Rearchitecting a Neuromorphic Processor for Spike-Driven Brain-Computer Interfacing

Hunjun Lee*
Hanyang University
Seoul, South Korea
hunjunlee@hanyang.ac.kr

Yeongwoo Jang
Seoul National University
Seoul, South Korea
yeongwoo.jang@snu.ac.kr

Daye Jung
Seoul National University
Seoul, South Korea
daye.jung@snu.ac.kr

Seunghyun Song
Seoul National University
Seoul, South Korea
seunghyun.song@snu.ac.kr

Jangwoo Kim⋆
Seoul National University
Seoul, South Korea
jangwoo@snu.ac.kr

2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO) | 979-8-3503-5057-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/MICRO61859.2024.00082

***Abstract*—Brain-computer interfaces (BCIs) are electrophysiological devices (e.g., electrode arrays) that connect the brain to a computer. They offer neuroscientific and neurological innovations by utilizing a dedicated processor for continuous BCI signal processing. Recent studies propose a scaled-up BCI that adopts an order of magnitude larger number of electrodes to more precisely interface with the brain. As the BCI scales, utilizing a spike-driven processor emerges as an alternative processing method, where the BCI offloads computations to the processor upon detecting spikes. However, the processor design for spike-driven processing has been relatively unexplored compared to that of the continuous processor.**

In this work, we propose NeuroLobe, a flexible and efficient processor design for spike-driven processing. The key idea is to utilize a neuromorphic processor to take advantage of its event-driven computing nature. We carefully rearchitect the existing neuromorphic system for the purpose of flexibly and efficiently deploying the BCI algorithms. First, we extend the instruction set architecture of the existing neuromorphic processor to flexibly deploy representative spike-driven BCI algorithms. Second, we redesign the connection controller and execution path to improve the performance. Third, we design a custom synchronization unit for scalable processing. Fourth, we implement a custom software stack to minimize load imbalance among the cores. Lastly, we design a multitask controller to simultaneously process multiple algorithms. We evaluate NeuroLobe on four representative BCI algorithms with 11 configurations. Evaluation results show that NeuroLobe surpasses CPU and GPU in terms of speed and energy efficiency.

***Index Terms*—Brain-Computer Interface, Spike, Neuromorphic Processor, Flexibility.**

I. Introduction

Brain-computer interfaces (BCIs) are electrophysiological devices that record and stimulate neurons in the brain. Among various BCI implementations, utilizing an implantable electrode array (i.e., implantable BCIs) offers high signal fidelity and spatio-temporal resolution to more precisely interface with the brain. When combined with a dedicated processor that decodes neural activity in the brain, they open up a broad range of opportunities to demystify the brain and treat neurological disorders [60], [120], [140]. For example, neuroscientists analyze BCI signals to understand memory formation [14], [18] and learning [87] in the brain. As another example, recent studies demonstrate brain-controlled robots by decoding BCI signals [50], [97].

Recently, the implantable BCIs have successfully scaled up to simultaneously record and stimulate a larger number of neurons for advanced applications. The cognitive functions in the brain involve population-wide activity of the neurons, and thus the larger-scale recordings result in a more precise readout of the brain's functional states [13], [77], [95], [139]. This motivates a scaled-up BCI that can record an order of magnitude larger number of neurons [35], [148].

Prior works employ a continuous signal processing strategy for the implantable BCIs. Some studies utilize BCIs with processing capabilities (on-BCI processing) [15], [60], [62], [120], [121], [152], [152]. They directly process continuous signals from BCI electrodes at the implant site, targeting latency-critical applications [120], [133]. Other works offload computations to an external processor (off-BCI processing) [4], [42], [108], [134], [137]. They rely on a wireless signal transmitter to continuously deliver all the recorded signals to an external device [146].

However, continuous signal processing potentially violates the thermal budget in scaled-up BCIs. As the BCI scales, the power consumed on computation increases accordingly [60]. Similarly, off-BCI processing incurs high communication overhead as the volume of the recorded signals increases. In both cases, the BCI system risks overheating the implant site when processing signals from scaled-up BCIs, which leads to permanent brain damage.

In contrast, this work focuses on designing a spike-driven off-BCI processor to cope with the tight thermal budget. To mitigate the high communication overhead, recent studies propose to decode only spiking activity in the recorded signals [19], [29], [85], [134]. It mitigates the communication overhead at the implant site by selectively sending spatially

*This work was done while the author was at Seoul National University.
⋆Corresponding author.

979-8-3503-5057-9/24/$31.00 ©2024 IEEE
DOI 10.1109/MICRO61859.2024.00082

and temporally adjacent BCI signals only upon detecting spikes [31], [46], [113]. In turn, the off-BCI processor executes dedicated algorithms (i.e., spike-driven BCI algorithms) to handle the spike-driven BCI signals.

Unfortunately, existing processors are either inefficient or inflexible to process various spike-driven BCI algorithms. To support a wide range of BCI applications, existing works rely on general-purpose processors such as CPUs [34], [42], [79], [112] or GPUs [135]–[137]. However, they are slow and inefficient when processing signals from scaled-up BCIs, limiting their usage in latency- and power-sensitive applications. Alternatively, other systems utilize custom-designed accelerators specialized for a specific BCI algorithm (e.g., template matching [2], neural network [140], spike sorting [130]). However, they are not applicable if the target application requires support for a wide range of algorithms.

We address the problem based on the observation that we can generalize the spike-driven BCI algorithms as a network structure where the nodes interact in an event-driven manner. *This observation motivates us to utilize a neuromorphic processor, which is an asynchronous manycore architecture with large on-chip memories to enable efficient event-driven processing of neural networks.* However, conventional neuromorphic processors are tailored to execute neural networks, making them unsuitable to handle other BCI algorithms.

In this work, we present NeuroLobe, a neuromorphic processor for spike-driven brain-computer interfacing. We extensively analyze various BCI algorithms to identify five unique challenges in utilizing neuromorphic processors for BCI workloads. Accordingly, we carefully rearchitect the state-of-the-art neuromorphic system [26], [27] to tackle each challenge. First, we extend the instruction set architecture (ISA) of the existing neuromorphic processor to flexibly support a wide range of algorithms. Second, we design a dedicated connection controller and pipelined execution path to accelerate common computation patterns in BCI algorithms. Third, we design an advanced synchronization unit to support complex synchronization patterns involved in BCI algorithms. Fourth, we implement a custom software stack to minimize the load imbalance among the cores. Finally, we design a multitask controller to simultaneously execute multiple algorithms.

We deploy four representative BCI algorithms with 11 configurations on NeuroLobe to demonstrate its flexibility and performance benefits. Experiments show that NeuroLobe is 69.91× faster and 1617.28× more energy-efficient compared to the software baseline which selectively adopts the CPU and GPU based on the efficiency. We also evaluate NeuroLobe against the tensor processor which is another potential candidate to enable efficient BCI processing. We configure sparse tensor accelerators to deploy the BCI algorithms and estimate their performance using Sparseloop [143]. The result shows that NeuroLobe achieves 82.50× and 264.49× superior energy-delay product over the sparse tensor accelerators based on the systolic array and vector unit, respectively.

In summary, we make the following contributions.

- **Spike-Driven BCI Processor Design:** As BCIs scale, spike-driven brain-computer interfacing emerges as a promising option. This work is one of the pioneering studies that addresses this timely topic using a novel processor design.
- **Rearchitecting a Neuromorphic Processor:** This work analyzes various aspects of neuromorphic processors when deploying non-neural network algorithms. Our analysis and the architectural techniques provide future research guidelines in utilizing neuromorphic processors for other workloads.
- **NeuroLobe Architecture:** We present NeuroLobe, a highly flexible and efficient spike-driven BCI processor. NeuroLobe achieves higher speed and energy efficiency over general-purpose processors and sparse tensor accelerators while supporting representative spike-driven BCI algorithms.
- **Software Artifacts:** We release our software artifacts, including the software stack, example programs, cycle-accurate simulators, and software baselines, to the community for further research.[1]

II. Background

A. Scaled-Up Implantable Brain-Computer Interface

BCIs have evolved to capture high-quality signals from a large number of neurons. Various BCI implementations exist, including functional magnetic resonance imaging (fMRI) [109], [117], [147], electroencephalogram (EEG) [7], [40], [90], [129] and microelectrode arrays [9], [23]–[25], [68]. Among them, implantable microelectrodes directly access the brain with minimal interference to achieve superior signal fidelity and resolution. Moreover, recent studies demonstrate scaled-up electrode arrays capable of interfacing with a large number of neurons. For example, Neuralink presents BCIs with 1,024 electrodes [148] and obtained Food and Drug Administration (FDA) approval to begin human clinical trials [76]. The scaled-up implantable BCIs revolutionize neuroscientific research and medical treatments by extensively and precisely interfacing with the brain [54]. However, they impose a strict thermal budget as dissipating excessive heat at the implant site potentially damages the brain.

B. Local Field Potential vs. Action Potential

The implantable BCIs can record two types of brain signals: local field potentials (LFPs) and action potentials (APs) [33], [35], [148]. LFPs reflect aggregated subthreshold activities to represent network-wide dynamics [89], [150]. This motivates the use of LFPs in various medical-purpose applications to detect abnormal population-wide oscillatory behaviors (e.g., seizure [81], [151], Parkinson's disease [125], [154], retinal degeneration [47]). On the other hand, APs represent spiking activities of individual neurons. APs contain more information about the movement intent compared to LFPs [38], and thus are widely used in various movement decoding tasks [19], [21], [29], [85], [134]. Also, APs are used in applications that require single neuron activities, such as an artificial retina [69], [114] or biological network reconstructions [49], [67].

[1] https://github.com/SNU-HPCS/NeuroLobe

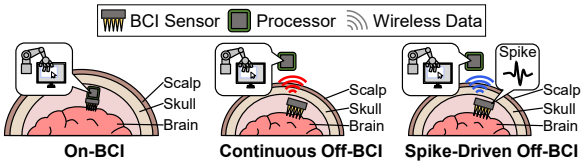


Fig. 1. An overview of the representative BCI signal processing methods. They differ in where the processor is placed and how the recorded signals are transferred.

C. BCI Signal Processing Methods

We categorize the representative BCI processing methods into three categories: On-BCI, continuous Off-BCI, and spike-driven Off-BCI (Figure 1). Each method varies in terms of computation overhead at the implant site, communication overhead, and the types of BCI signals they can handle (Table I).

1) On-BCI: On-BCI processing utilizes a processor implanted with BCI electrodes inside the brain (`On-BCI` in Figure 1 and Table I). It enables untethered operations without an external processor. This simplifies the overall processing pipeline and eliminates the need to transfer the BCI signals out of the brain. However, on-BCI processing should keep the power consumed on processing within a tight thermal budget [60]. As a result, existing works design an extremely low-power accelerator for a specific BCI algorithm [62], [96], [121], [152], [154]. Notably, HALO [60] proposes an accelerator-rich system to support various BCI algorithms.

2) Off-BCI: Off-BCI processing offloads computations to an external processor by transferring the BCI signals over the skull. Therefore, the processor can consume relatively high power compared to on-BCI processing. The computation overhead increases linearly (e.g., SVM) or super-linearly (e.g., cross-correlation) with the number of electrodes [60], making off-BCI processing promising for scaled-up BCIs.

There are two variants of the off-BCI processing method: continuous off-BCI and spike-driven off-BCI. First, continuous off-BCI processing transfers all the BCI signals to an external processor [41], [110], [115] (`Continuous Off-BCI` in Figure 1 and Table I). It enables the external processor to support both LFP- and AP-driven BCI applications. However, this comes at the cost of high communication overhead (e.g., power consumption, RF energy) [146]. On the other hand, spike-driven off-BCI processing relies on the compressive nature of APs to minimize the communication overhead [38], [45], [80], [92], [94], [148] (`Spike-Driven Off-BCI` in Figure 1 and Table I). Instead of continuously transferring all the BCI signals, it transfers the signals based on the peaks in the APs (i.e., spikes). Despite its limited coverage, it greatly reduces the computation and communication overhead at the implant site. Therefore, it is widely adopted in scaled-up BCIs with a large number of electrodes [17], [38], [55], [66], [148].

D. Representative Spike-Driven BCI Algorithms

We conduct a thorough study on various BCI applications (e.g., medical purpose, neuroscientific analysis) in implantable microelectrode-based BCI systems. Then, we observe that the BCI applications consist of four representative spike-driven BCI algorithms (Figure 2). They are spike sorting [20], [46], [83], [103], [106], [111], neural network [16], [29], [84], [108], [118], [135], template matching [22], [53], [61], [64], [82], [127], and pairwise correlation [36], [39], [70], [98], [105], [128], [138]. We describe the algorithms using a network structure.

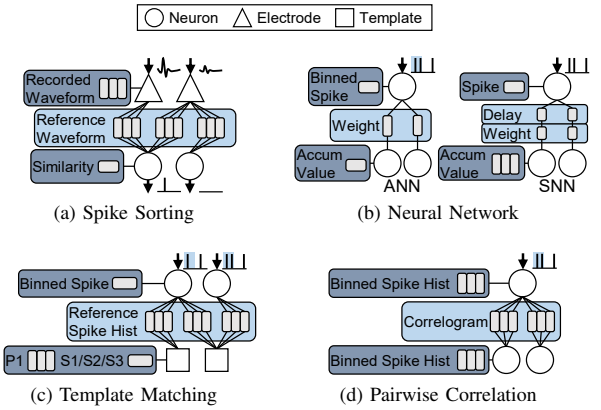
TABLE I
COMPARISONS BETWEEN THE BCI SIGNAL PROCESSING METHODS. THE SPIKE-DRIVEN OFF-BCI PROCESSING IS PROMISING FOR SCALED-UP BCIS DUE TO THE LOW COMPUTATION AND COMMUNICATION OVERHEAD.

	Overhead @ Implant Site		Coverage
	Computation	Communication	
On-BCI	High	Low	AP/LFP
Continuous Off-BCI	Low	High	AP/LFP
Spike-Driven Off-BCI	Low	Low	AP

1) Spike Sorting (SS): Spike sorting is used to distinguish the neurons from the spike waveforms recorded on the BCI electrodes (Figure 2a). It is widely adopted as the first step in BCI signal processing in various applications [46], [106]. Spike sorting [32], [144] keeps reference waveforms of the adjacent electrodes for each neuron. Upon detecting a peak in an electrode, it compares recorded waveforms from adjacent electrodes with the reference waveforms of each neuron.

Spike sorting is represented as a network of electrodes and neurons where the edges connect adjacent neurons and electrodes, and store the reference waveforms [32]. First, upon receiving spatially and temporally adjacent BCI signals, the electrodes update the recorded waveforms (❶). Then, each electrode detects a peak in the updated waveforms (❷). Upon detecting a peak, the electrode traverses the connected neurons (❸). In turn, each neuron traverses the connected electrodes to partially calculate the similarity between the reference waveforms on the edges and the recorded waveforms on the electrodes (❹). Next, the neurons accumulate the partial similarity (❺) and check if the similarity exceeds a threshold (❻). In that case, the neurons traverse the electrodes to subtract the reference waveforms from the recorded waveforms (❼).

2) Neural Network (NN): BCI applications adopt neural networks for accurate signal classification tasks such as movement decoding [134], [140]. Neural networks can be classified into two types: artificial neural network (ANN) and spiking neural network (SNN). An ANN consists of a network of neurons with dedicated input neurons that receive spikes from BCIs (or sorted spikes in `SS`) (Figure 2b). Upon receiving spikes from the BCIs, the input neurons bin the spikes (❶) over a time interval. Then, each input neuron iterates over the connected neurons to transfer the binned spike if it is non-zero (❷). Then, the destination neurons multiply the binned spikes with the weight for each edge and accumulate the value (❸). Finally, each neuron applies an activation function to the accumulated value and transfers the value to the connected neurons if it is non-zero (❹). An SNN operates in a slightly different manner by updating the state of each neuron and transferring a spike after a delay if it exceeds a threshold (❹).



(a) Spike Sorting (b) Neural Network (c) Template Matching (d) Pairwise Correlation

Fig. 2. Network representations of the spike-driven BCI algorithms.

3) Template Matching (TM): Template matching detects recurrences of meaningful spike patterns in the brain [22], [53] (Figure 2c). It compares the spike history with the reference spike history (template) for a time interval (template width) binned over a time window [2], [37], [82]. As a comparison metric, it typically adopts Pearson's Correlation Coefficient (PCC) [2]. To efficiently calculate the PCC, it calculates the (1) dot product between the binned spike history and reference binned spike history (S1), (2) sum of binned spikes (S2), and (3) sum of squared binned spikes (S3).

We represent template matching as a network of neurons and templates following the existing template matching accelerator [2]. First, the neuron bins the spikes from BCIs (or sorted spikes) (❶). If the binned spike is non-zero, it traverses the connected templates to transfer the spike (❷). Upon receiving a spike, each template updates S1, S2, and S3 (❸). For S1, each template keeps a list of partially evaluated S1 (i.e., P1) over the template width, using a sliding window methodology. It sequentially multiplies the spike with the reference history and accumulates it to P1. Lastly, each template calculates the PCC using S1, S2, and S3 (❹).

4) Pairwise Correlation (PC): Pairwise correlation calculates the correlation between pairs of neurons to understand the functional and physical connectivity of the brain [3], [58], [67] (Figure 2d). It builds a histogram of the time interval between the binned spikes of a source neuron and a destination neuron (i.e., correlogram). Then, it uses the correlogram to evaluate the correlation between the connected neurons.

Pairwise correlation is represented as a network of neurons where the edges keep the correlogram between the connected neurons [58]. First, each neuron updates the binned spike history and accumulates the squared binned spike from BCIs (or sorted spikes) (❶). If the binned spike is non-zero, it traverses the target neurons to transfer the binned spike (❷). Upon receiving a spike, the target neurons iterate over the spike history to count the number of spike pairs for each time interval. Then, they accumulate the results to the correlogram (❸). Finally, the algorithm periodically measures the correlation between the neurons by traversing all the edges (❹).

E. Neuromorphic Processor

A neuromorphic processor is a brain-inspired computing architecture that enables low-power and massively parallel execution of ANNs [91], [101] and SNNs [5], [27]. It intertwines compute units with large on-chip memory units, eliminating the need for off-chip memory accesses. In addition, it employs an asynchronous manycore structure for efficient event-driven computations. Each core handles a dedicated set of neurons and the fan-in synapses (i.e., edges) of the neurons. First, it performs neuron computation to update the neurons. If a neuron fires a spike, the core transfers the spike to the destination cores which handle the destination neurons. After receiving the spike, the destination cores perform spike processing to update the destination neurons using the weight and delay of the synapses.

Figure 3 illustrates a neuromorphic processor similar to the Loihi series [26], [27][2]. The Loihi series is the state-of-the-art neuromorphic processor, utilizing a 2D-mesh interconnection network. Each core keeps the network data using the heterogeneous memory units: state, connection, and history memory. The state memory stores the internal states of the dedicated neurons. The connection memory stores the synapses and the connection controller generates the memory addresses of the connected synapses. The history memory is a circular queue that accumulates the weight for spike processing with delay.

The Loihi series adopts programmable datapaths to flexibly support the neural networks. It adopts separate datapaths for the spike processing and neuron computation. For spike processing, it relies on the microcode-programmable datapath that supports sum-of-product operations [27]. It adopts a more flexible neuron computation datapath that supports basic ALU, spike trigger, control, and simple memory instructions [26].

At the end of each timestep, the Loihi series performs a barrier synchronization to ensure that all the spikes are delivered to the destination cores [27]. A representative way to implement barrier synchronization is the acknowledgment-based method [73]. The cores perform neuron computation and transfer spikes to the destination cores. Then, they wait for the acknowledgment packets (ACKs) from the destination cores (`Ack Collection`). After finishing neuron computation and receiving the ACKs, the cores handshake sync packets with their neighbors (`Barrier`). After the cores synchronize, they process all the incoming spikes from the previous timestep and proceed to the next timestep.

III. Motivation & Challenges

A. Workload Analysis

Based on the network representation of the spike-driven BCI algorithms, we classify the computations into three stages: pre-processing, event-driven, and post-processing (Figure 4a). First, pre-processing indicates the computations which iterate over the nodes to determine if a node triggers an event. For

[2]Loihi 2 does not provide in-depth explanations on the detailed working mechanism. Therefore, we use the term Loihi series to indicate Loihi 1 [27] integrated with a programmable neuron computation datapath in Loihi 2.

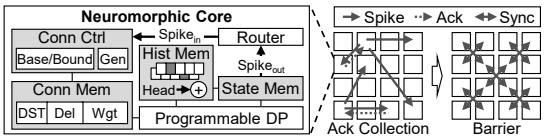
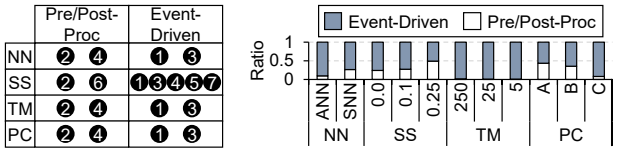


Fig. 3. An internal architecture of the representative neuromorphic processor similar to the Loihi series.



(a) Classification (b) Latency Breakdown

Fig. 4. Classification of the computation stages and CPU latency breakdown of four representative spike-driven BCI algorithms.

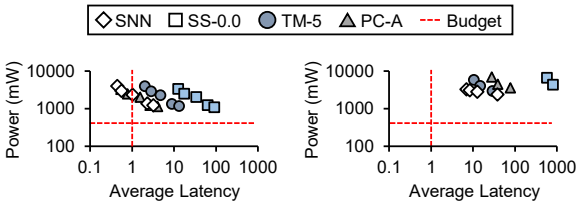
example, updating internal states in neural networks (NN-❹) falls into this category. Second, event-driven refers to the event-triggered computations between the nodes such as accumulating correlograms (PC-❸). Lastly, post-processing indicates the computations after the event-driven interactions such as calculating PCCs (TM-❹).

We profile four representative algorithms with 11 configurations on CPU to analyze the performance bottleneck (evaluation setup in Section V-A). The result shows that the time consumed on the event-driven computation stages becomes a critical bottleneck (Figure 4b). Using workloads with a smaller number of connections (e.g., SS-0.0 vs. SS-0.25) reduces the event-driven computation overhead to some extent; however, they still account for more than 50% of the total latency. As a result, efficiently processing the event-driven computation stages is a key to the scaled-up BCI.

B. Limitations of the Existing Systems

To realize a practical BCI system, the processor should be fast and energy-efficient [2], [51], [56], [88] while being flexible to support representative algorithms with different configurations [10], [63], [78], [132]. However, existing works fail to both efficiently and flexibly support the event-driven computation stages in the BCI algorithms. The pre/post-processing stage involves iterating over all the nodes to exhibit a statically fixed computation pattern. Therefore, they can be accelerated using the SIMD architectures (e.g., GPUs). On the other hand, the event-driven computation pattern changes dynamically, and thus it is not easily applicable to conventional architectures. As a result, they potentially violate the speed and efficiency requirements of the BCI applications. For higher performance, other systems utilize a hardwired accelerator for a specific algorithm (e.g., neural network [140], template matching [2]). Unfortunately, they are not applicable if the target application requires support for multiple algorithms.

We evaluate the performance of general-purpose processors when deploying the spike-driven BCI algorithms by varying the number of cores and clock frequencies (evaluation setup in Section V-A). The results illustrate the limitations



(a) CPU (b) GPU

Fig. 5. The performance limitations of the CPU- and GPU-based BCI systems. They fail to process the BCI signals in real-time using a portable power source.

of CPU/GPU-based BCI systems (Figure 5). We assume the processor is powered by a portable battery with a capacity of 10,000 mWh, similar to modern smartphones [102]. Also, it should operate for at least 24 hours on a full charge [93], [116], which limits the average power consumption to 420 mW. In addition, the system should operate in real-time, with average latency smaller than the interval between consecutive BCI signals (e.g., 1 ms in SNN, 5 ms in TM). The results show that both CPU (Figure 5a) and GPU (Figure 5b) fail to meet both latency and power budgets. In some configurations, they meet the latency budget by increasing the number of cores and clock frequency; however, this results in significantly higher power consumption.

C. Utilizing Neuromorphic Systems for BCI Algorithms

This work proposes to utilize neuromorphic systems to deploy the entire BCI application chains, encompassing both neural networks and signal processing. The signal processing in the spike-driven BCI algorithms (i.e., SS, TM, and PC) share common computational features with the neural networks, such as ANNs and SNNs. They are both represented as network structures (Section II-D), and the event-driven interactions between the nodes dominate the overall computations (Section III-A). Therefore, we generalize the state-of-the-art Loihi-style neuromorphic system to flexibly deploy the spike-driven BCI algorithms in the following manner.

Mapping. We distribute the nodes (e.g., neurons, templates) and their fan-in edges that comprise the target BCI algorithm to each core. We place (1) time series data in the nodes (e.g., waveforms in SS) into the history memory, (2) non-time series data in the nodes (e.g., S1-S3 in TM) into the state memory, and (3) data in the edges into the connection memory.

Computation. We utilize the spike processing and neuron computation datapaths in the Loihi series as a starting point to support the event-driven computation and pre/post-processing stages, respectively. The event-driven computation stage resembles spike processing as both are triggered by events (or spikes) and utilize the data in the fan-in edges (or synapses) and destination nodes (or neurons). Similarly, the pre/post-processing stage resembles neuron computation in that both iterate over nodes and conditionally trigger events (or spikes).

Synchronization. We modify the synchronization mechanism to (1) differentiate BCI signals from each timestep and (2) support multiple stages of pre/post-processing and event-driven computation to process BCI signals from each timestep.

TABLE II
CHALLENGES IN UTILIZING THE STATE-OF-THE-ART NEUROMORPHIC SYSTEMS TO DEPLOY THE BCI ALGORITHMS.

Type		Existing Neuromorphic System	NeuroLobe
Flexibility	Single Stage	△ (Limited ISA)	Sec. IV-A1
	Multiple Stages	×	Sec. IV-A2, Sec IV-D1
Performance	Connection Ctrl	△ (Limited Type)	Sec. IV-B
	Datapath	△ (NN Only)	Sec. IV-C
Scalability	Cascaded Sync	×	Sec. IV-D2
	Load Balancing	×	Sec. IV-E2
Multitask	Spatio-Temporal	×	Sec. IV-F
Usability	Low-Level Programming I/F	◯ (Low-Level Programming I/F)	Sec. IV-E1

The cores receive BCI signals in the form of event packets and perform event-driven computations upon receiving them. Then, they synchronize to trigger the following pre/post-processing stage. During the pre/post-processing, they trigger a subsequent event-driven computation stage by sending packets. This procedure is repeated until the last stage, after which the cores synchronize and wait for BCI signals for the next timestep.

D. Challenges in Utilizing Neuromorphic Systems

Unfortunately, the Loihi series and its variants [5], [6], [43], [65], [72], [73], [101], [104] are tailored to execute neural networks, limiting their usage in the BCI context (Table II).

1) Flexibility: The BCI algorithms consist of a sequence of heterogeneous pre/post-processing and event-driven computation stages. Therefore, the processor should flexibly support each computation stage and various sequences of the stages.

The reconfigurable datapaths in the Loihi series provide limited flexibility to program each computation stage. Unlike the neuron computation datapath which supports basic arithmetic, control, and memory instructions, the spike processing datapath is specialized to support sum-of-product operations using the data in the connection memory. However, the event-driven computation stages demand a wider range of dataflows, including conditionally triggering events and accumulating data to the register file (Refer to Section IV-C). In addition, the neuromorphic processors support a single-stage pre/post-processing and event-driven computation. It adopts a simplified instruction controller and single-phase synchronization unit that fails to deploy multiple computation stages.

2) Performance: The BCI algorithms exhibit connection and computation patterns that are not observed in neural networks. Therefore, it is hard to directly utilize the performance optimization methods in the existing neuromorphic processors.

The neuromorphic processors adopt a connection controller to generate the memory addresses for the connection patterns in neural networks (e.g., convolution, weight-delay pair) [26], [65]. However, the BCI algorithms exhibit complex connection patterns. For example, the connections between the neuron and template in TM keep a reference spike history of an arbitrary length. This complicates how the connection data are stored, demanding the need to redesign the connection controller.

Prior studies propose a fully pipelined datapath to accelerate common computation patterns in SNNs [6], [71]. They integrate the pipelined datapaths to the programmable neuromorphic processors (e.g., Loihi) to offload a portion of the computations. However, the existing works are insufficient to support common computation patterns in the BCI algorithms, rendering them inapplicable to non-neural network algorithms.

3) Scalability: We observe two performance issues when deploying BCI algorithms on the manycore neuromorphic system. First, it suffers from redundant synchronizations when an event-driven computation stage triggers another event-driven computation stage (i.e. cascaded events). The neuromorphic processors can synchronize a single type of event at a time, and thus, it incurs additional synchronization between the cascaded events. For example, for the cascaded event-driven computation stages in SS (❸–❺), it synchronizes three times instead of synchronizing at once. Second, the processor suffers from load imbalance if the number of a specific type of node is smaller than the total number of cores. For instance, there are only a small number of templates in TM. In such cases, the processor cannot fully distribute the computations for the templates to the cores, limiting the scalability.

4) Multitask: The existing neuromorphic processors are primarily designed to execute a single algorithm. However, users often require simultaneous execution of multiple algorithms in practical use cases. For example, recent studies propose multitasking in the BCI-driven motor imagery and visuospatial attention task [44], [153]. Other works utilize spike sorting and template matching combined with neural networks in movement decoding and seizure detection [10], [78].

5) Usability: The users need a software stack that enables them to flexibly program emerging BCI algorithms using a low-level programming interface. We follow the low-level programming interface such as Lava [57] to guarantee high programmability of the system.

IV. NEUROLOBE ARCHITECTURE

In this section, we present NeuroLobe, a flexible and efficient BCI acceleration system (Figure 6). It consists of three components. First, it utilizes a software stack to deploy the BCI algorithms. Second, it keeps an external interface that wirelessly receives BCI signals from the implanted BCIs and transfers the computation results to other devices (e.g., robotic arm, medical alert system). Lastly, it uses the 64-core Loihi-style processor for BCI signal processing.

Compile time. The software stack compiles the user-defined BCI algorithms at the host machine to extract network, routing, and synchronization metadata. Also, it converts the user-level instructions to the NeuroLobe ISA. The host machine transfers all the compiled results to the external interface at the NeuroLobe hardware which transfers them to each core via 2D mesh NoC.

Runtime. At runtime, NeuroLobe receives the BCI signals using the external interface. It temporarily stores the signals

TABLE III
OVERVIEW OF THE NEUROLOBE INSTRUCTION SET. THE SHADED ROWS INDICATE ADDITIONAL PERFORMANCE-OPTIMIZING INSTRUCTIONS.

Instruction	Description	Operands
ALU	Fixed/floating point arithmetic/logical operation	func type, src1/2/3, dest, immediate
Event Trigger	Conditionally trigger/probe event (eq, neq, ge, gt, le, lt)	func type, src1/2, immediate, node ID, data reg, event type
Cond Jump	Conditionally jump (eq, neq, ge, gt, le, lt)	func type, src1/2, immediate, jump addr
Memory	Memory access (connection, history, state)	memory/access type, read/write/address reg, mem offset
Loop Node	Loop through the nodes	loop reg, node type
Loop Conn	Loop through the connections for an incoming event	loop reg, base reg, conn type
Connection Ctrl	Initialize the connection controller (Section IV-B)	connection pattern
Pipelined Execution	Execute the pipelined execution path (Section IV-C)	dataflow types, function type, mem offset, reg addr

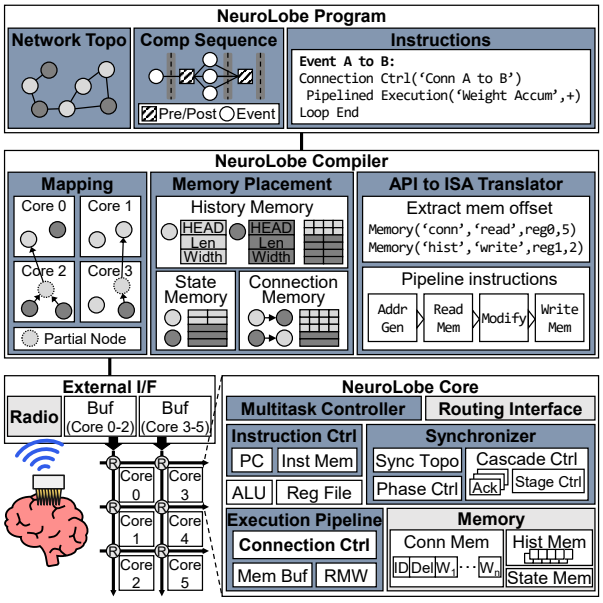


Fig. 6. An overview of the NeuroLobe system as a result of our rearchitecting procedure. The modified components are colored in dark blue.

in the buffers that are connected to each column of the NeuroLobe cores. After the cores synchronize, the buffers transform the signals into packets and transfer them to the cores via the 2D-mesh NoC. In turn, each core processes the signals from a dedicated set of electrodes and synchronizes. After finishing all the computation stages for the given BCI signals, the cores synchronize with the external interface.

We rearchitect the Loihi-style neuromorphic system to solve the limitations when deploying the BCI algorithms (Table II). First, we extend the Loihi2-style ISA for higher flexibility (Section IV-A1) and design a multi-stage instruction controller (Section IV-A2). Second, we design a versatile connection controller for the BCI algorithms (Section IV-B). Third, we identify common computation patterns to design a pipelined execution path (Section IV-C). Fourth, we propose a synchronization unit with multi-stage (Section IV-D1) and cascaded synchronization support (Section IV-D2). Fifth, we develop a low-level programming interface (Section IV-E1) and a software optimization technique to minimize load imbalance (Section IV-E2). Lastly, we devise a multitask controller for efficient multitasking (Section IV-F).

A. Instruction Controller

1) ISA Extension: We extend the Loihi 2 ISA [26] to flexibly support both pre/post-processing and event-driven computation stages in the BCI algorithms (white rows in Table III). Following the high-level descriptions of the instruction set in the Loihi 2 technical report [26], we model four basic Loihi 2-style instructions for the neuron computation (i.e., `ALU`, `Event Trigger`, `Cond Jump`, and `Memory`). Then, we extend the ISA to support both pre/post-processing and event-driven computation stages. We introduce loop instructions (`Loop`) that allow users to describe both types of stages. Therefore, both stages can use the basic ALU, control, and memory instructions in Loihi 2 which were only applicable to the pre/post-processing stage (neuron computation). Then, we extend the memory instruction to access heterogeneous memory units (i.e., connection, history, state memory) with heterogeneous data types.

ALU. NeuroLobe supports various fixed and floating-point operations, ranging from simple (e.g., addition, multiplication, and comparison) to complex (e.g., division and square root) operations. It also supports conversion between fixed and floating-point data. To minimize the area and power consumption, the ALU consumes multiple cycles (i.e., 12 cycles) to execute divide and square root operations.

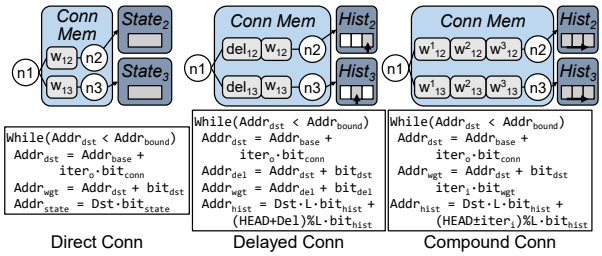
Event Trigger. NeuroLobe enables custom event-trigger operations that conditionally trigger event-driven computations (e.g., peak detection in `SS`-❷). The instruction specifies the node ID (`node ID`) that triggers the event and the required data (`data reg`) to trigger the event-driven computations.

Cond Jump. NeuroLobe supports conditional jump to jump `immediate` number of instructions if a condition is met.

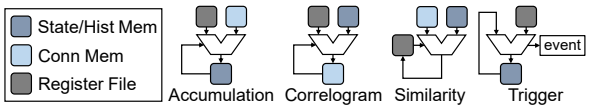
Memory. NeuroLobe enables fine-grained memory access using register-indirect addressing (`address reg`) for the connection, history, and state memory. Additionally, it can manage multiple data types (e.g., S1, S2, and S3 in `TM`) within the same memory units using `mem offset`.

Loop Node. The Loop Node instruction is used to describe pre/post-processing that iterates over the nodes. For each iteration, the instruction stores the node ID in the register (`loop reg`), allowing subsequent instructions to read and write the data from/to the memory units using the node ID.

Loop Conn. The Loop Conn instruction is used to describe event-driven computations that iterate over the connections.



(a) Connection type



(b) Dataflow type

Fig. 7. The list of common connection and dataflow types in the BCI algorithms when designing the connection controller and execution path.

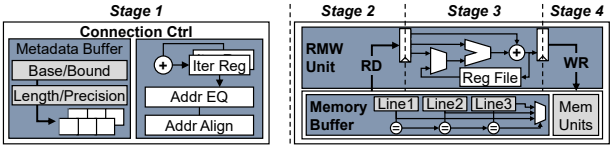
The instruction uses `loop reg` to track the iteration count and `base reg` to track the base address of the connection memory. Accordingly, the subsequent instructions use the registers to address and access the connection memory.

2) Multi-Stage Instruction Support: We extend the instruction controller to support multiple stages of pre/post-processing and event-driven computation. To distinguish event-driven computations at different stages, NeuroLobe embeds event types in the packets. In turn, the instruction controller uses the event type to fetch the target instructions. For example, when deploying `NN`, the controller fetches the instructions for binning upon receiving a packet for a BCI signal (`NN-`❶). Conversely, it fetches the instructions for weight accumulation (`NN-`❸) when receiving a packet for a spike. The instruction controller also manages executing the target pre/post-processing stage after the synchronization.

B. Versatile Connection Controller

We develop a versatile connection controller after identifying three representative connection patterns in the BCI algorithms: `Direct`, `Delayed`, and `Compound` (Figure 7a). The controller employs a reconfigurable datapath that supports the representative connection patterns; therefore, NeuroLobe can offload memory address generation to the controller, effectively mitigating the overhead. Direct connection exclusively stores the destination ID (Dst) and weight (w), utilizing Dst to index the state memory. Delayed connection keeps Dst, w and delay (Del) of each connection, accessing the history memory by adding Del to the head address ($Head$) (Section II-E). Compound connection stores a list of weights (or values) such as the reference waveform in `SS`. It sequentially accesses each weight and history memory in a sliding manner.

The controller sequentially iterates over the connections for the incoming event ($iter_o$), and iterates over the weights associated with each connection ($iter_i$) (Refer to the pseudocode in Figure 7a). For each iteration, it generates the memory addresses of the state memory ($Addr_{state}$), history memory ($Addr_{hist}$), and each datatype in the connection memory



(a) Connection controller (b) Read-modify-write unit

Fig. 8. An internal architecture of the optimized execution path to support common connection and dataflow type in the BCI algorithms.

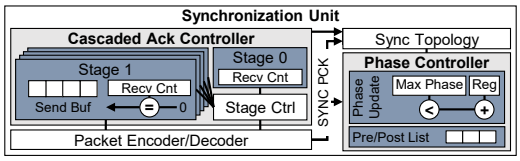
($Addr_{dst,del,wgt}$). Note that L indicates the length of the history memory, $Addr_{base,bound}$ indicates the base and bound address, and bit refers to the bit precision. After evaluating all the addresses, it aligns the memory addresses according to the line width of each memory unit.

Figure 8a illustrates an overview of the connection controller. After receiving an event, the controller defines the connection pattern using `Connection Ctrl` instruction in Table III. Then, the controller accesses the base and bound address, history length, and bit precision for the connection and temporarily stores the data in the 64-entry `Metadata Buffer`. After initializing the metadata, the controller increments the iteration count and generates memory addresses for each memory unit. For each iteration, it utilizes the datapath to generate memory addresses for the target connection pattern. Finally, the controller transfers the addresses to the execution path to read, modify, and write the data.

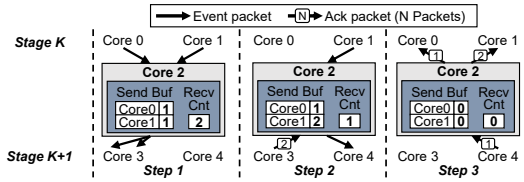
C. High-Speed Execution Path

NeuroLobe offloads common computation patterns in the BCI algorithms to the high-speed execution path. From our analysis, we identify representative computation patterns that include weighted accumulation (`Accumulation`), updating connection memory (`Correlogram`), accumulating computation results to the register file (`Similarity`), and triggering other events (`Trigger`) according to the incoming packets (Figure 7b). We generalize them using four computation stages. After receiving a packet, the core iterates over the connections by (1) using the connection controller to generate memory addresses for each connection, (2) loading data for each connection, (3) modifying the data using an arithmetic or event-trigger instruction, and (4) writing the modified data back to the memory. Then, we pipeline the four-stage computations based on the observation that the four-stage computations for each iteration target different connections, and thus, do not suffer from data hazards.

Figure 8 illustrates the pipelined execution path. It consists of a connection controller (Figure 8a) and a read-modify-write unit (`RMW Unit`) (Figure 8b). We design the RMW unit based on the commonly appearing computation patterns (Figure 7b) along with `Memory Buffer` which is a three-entry CAM to cache the memory lines. It eliminates redundant memory accesses if computations for consecutive connections access the addresses in the same memory line. NeuroLobe initiates the pipelined path using `Pipelined Execution` instruction (Table III) to configure the execution path.



(a) An overview of the advanced synchronization unit.



(b) Working mechanism of the cascaded ack controller

Fig. 9. Overview and mechanism of the advanced synchronization unit.

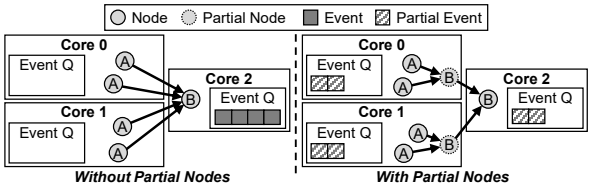
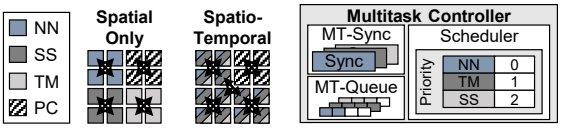


Fig. 10. An overview of the partial node insertion technique. The compiler inserts partial nodes for the connections between the nodes A and B.



(a) Multitasking overview (b) Multitask controller

Fig. 11. An overview of two multitasking schemes when deploying four algorithms on NeuroLobe and the internal architecture of the multitask controller.

D. Complex Synchronization Support

NeuroLobe supports complex synchronization patterns to accurately and efficiently deploy the BCI algorithms (Figure 9a). It adopts a phase controller to manage multiple synchronization stages. Also, it uses a cascaded ack manager to synchronize the cascaded event-driven computation stages.

1) Phase Controller: The phase controller maintains a phase update unit which updates the current phase after the synchronization (Figure 9a). Then, the phase controller schedules different pre/post-processing stage associated with the current phase from the `Pre/Post List`.

2) Cascaded Ack Controller: The cascaded ack controller supports an acknowledgment protocol for multiple stages of event-driven computations within a single synchronization phase. The controller manages a send buffer (`Send Buf`) and a receive counter (`Recv Cnt`) for each stage within the target phase (Figure 9a). Upon receiving an event packet, the core performs event-driven computations for the event. If the event triggers cascaded events, the controller increments the receive counter. Then, instead of sending an ACK packet to the source core, it increments the entry for the source core in the send buffer to keep the number of ACK packets to transfer to each core. Upon receiving an ACK packet, the controller decrements the receive counter. If the counter becomes zero, the controller iterates over the send buffer and sends the ACK packets embedded with the entry value for each source core.

Figure 9b illustrates a working mechanism. In the first step, Core-2 receives two packets that trigger *kth* stage event-driven computations (i.e., *kth* stage packet). After the core performs the event-driven computations triggered by each packet, it triggers *(k+1)th* stage events at Core-3. In this case, the controller increments the receive counter and send buffer according to the source core ID (i.e., Core-0 and Core-1) of the *kth* stage packets. In the next step, Core-2 receives *kth* stage packet from Core-1 to trigger a *(k+1)th* stage event at Core-4. Accordingly, the controller increments the entry for Core-1 in the send buffer and receive counter. It also receives an ACK packet from Core-3 embedded with the value of two, which decrements the receive counter by two. In the last step, Core-2 receives an ACK packet from Core-4, and the receive counter for *kth* event-driven computation stage becomes zero. Consequently, Core-2 sends ACK packets to Core-0 and Core-1 with the value of one and two, respectively.

E. Custom Software Stack

1) Front-End: NeuroLobe provides a low-level programming interface to define BCI algorithms with arbitrary network topologies, sequence of computation stages, and instructions for each stage (Figure 6-`NeuroLobe Program`). NeuroLobe compiler compiles the user-defined algorithms to program the memory, synchronization unit, and instruction controller. First, the users define the network with various APIs, including Pytorch [99] and NetworkX [48]. Then, they define the computation sequence using a dependency graph of computation stages. In turn, the compiler extracts the synchronization metadata. Lastly, the users define the pre/post-processing and event-driven computation stages using a low-level API that resembles the NeuroLobe ISA (Table III). The compiler translates the low-level API by (1) placing the network data to the memory units and setting the memory offsets, (2) allocating the register addresses, and (3) setting the jump addresses.

2) Back-End: The compiler back-end effectively handles the load imbalance by inserting a partial node, which offloads a portion of the computations to the source cores. Figure 10 illustrates an example of the partial node insertion. In the example, the compiler inserts partial nodes to the source cores (Core-0 and Core-1) for the edges connecting node-A and node-B. Then, it splits the computation A-to-B (i.e., computation of node-B triggered by node-A) into computations A-to-partial B and partial B-to-B. As a result, a portion of the event-driven computations is spread to Core-0 and Core-1, and the computation bottleneck at Core-2 is relieved.

F. Multitasking Support

NeuroLobe employs a multitask controller to both spatially and temporally multitask BCI algorithms (Figure 11). The figure illustrates an overview of the spatial multitasking and spatio-temporal multitasking when deploying four algorithms (Figure 11a). In the example, we assume that each algorithm requires at least four NeuroLobe cores to store the network data in the on-chip memories. NeuroLobe enables users to allocate a dedicated set of cores for each algorithm (spatial-only). In turn, each set of cores computes and synchronizes independently.

The spatial-only multitasking fails to allocate additional cores to high-priority or compute-intensive tasks in some cases. If the processor allocates an additional core to a high-priority task (e.g., `NN`), then it should allocate only three cores to a low-priority task (e.g., `SS`). In this case, the processor cannot store all the network data in the on-chip memories and fails to work properly. Therefore, NeuroLobe supports temporal multitasking using a multitask controller (Figure 11b) The multitask controller extends the synchronizer and event queue to support up to eight tasks. The synchronizer enables per-task synchronization with different topologies. Also, it schedules the events using the priority-aware scheduler.

However, the temporal multitasking adversely affects the overall performance if a certain task incurs long per-computation latency. For example, the post-processing in pairwise correlation involves iterating over all the components to calculate the cross-correlation (`PC`-❹), and thus incurs much longer latency compared to the other types of computations. Once the post-processing has been scheduled, the task scheduler cannot schedule events for other tasks until it finishes. As a result, the compiler prevents allocating a high- and low-priority task to the same core if the low-priority task exhibits high per-computation latency.

V. Evaluation

A. Implementation & Evaluation Setup

1) Implementation: We allocate an 8-kB SRAM to the state memory, 32-kB SRAM to the history memory, and a total of 400-kB SRAM to the connection memory for each NeuroLobe core. The cores communicate using a 72-bit packet that embeds the event type, task ID, source/destination core ID, and payload. To measure the overall energy efficiency and area of the NeuroLobe system, we write the RTL code for the NeuroLobe, including the external interfaces in Verilog. We synthesize the code using 45-nm NanGate FreePDK45 Library [123] assuming 400 MHz clock frequency with a 20% timing margin. Also, we obtain the SRAM and router link parameters using Accelergy [141]. The obtained parameters are scaled to 20 nm for fair comparisons against the CPU and GPU baselines [122]. For an end-to-end performance, we also implement the external interface that receives BCI signals using the wireless body area network (WBAN) transceiver [59] and interfaces with the cores. Finally, we implement an execution-driven simulator to evaluate the speed and energy efficiency of NeuroLobe for different architectural configurations. We have verified its functional correctness against the software implementation of each algorithm. Also, the timing model has been validated for each hardware unit by comparing the simulated performance against that of its RTL implementation.

TABLE IV
Configurations of the Evaluated BCI algorithms.

Name	Network Configuration		Dataset
NN-ANN	MLP (1600-800-2)	ANN, Bin: 120 ms	MAZE [21]
NN-SNN		SNN, Bin: 1 ms	
SS-0.0	Electrodes: 1600 Neurons Per Electrode: 0.65	Sparsify: 0.0	MEArec [12]
SS-0.1		Sparsify: 0.1	
SS-0.25		Sparsify: 0.25	
TM-250	Neurons: 1600 Templates: 16	Bin: 250 ms, Width: 5 s	BUMP [19]
TM-25		Bin: 25 ms, Width: 5 s	
TM-5		Bin: 5 ms, Width: 5 s	
PC-A	Neurons: 1600 Connections Per Neuron: 160	Bin: 1 ms, Period: 0.1 s	RTT [85]
PC-B		Bin: 10 ms, Period: 0.1 s	
PC-C		Bin: 10 ms, Period: 1 s	

2) Benchmark: We use four representative BCI algorithms with 11 configurations (Table IV). In the absence of a benchmark suite for scaled-up BCIs, we utilize Neural Latents Benchmark [100], the first open-source benchmark suite for neural spiking activities recorded on 96-electrode Utah Arrays. We adopt `MAZE`, `BUMP`, and `RTT` datasets which represent the most representative movement decoding task in the BCI applications. To generate datasets for scaled-up BCIs, we calculate a Gaussian rate template for each electrode [1]. Then, we randomly generate artificial spike trains from the templates through a non-stationary Poisson process [149]. For `MEArec` dataset, we use MEArec [12] to generate signals from 1,600 electrodes recording 1,041 neurons.

For `SS`, we use SpikeInterface [11] and Spyking Circus [144] to generate reference waveforms with varying sparsify (i.e., pruning) parameters. The radius and temporal width of the adjacent BCI signals are set to 250 um and 3 ms. We encode the BCI signals using a 12-bit header followed by the run-length encoded data. Also, we quantize the BCI signals to 5 bits, resulting in a less than 1% accuracy drop. For `NN`, we use Pytorch [99] to train the ANNs and Intel's Lava framework [57] to train the SNNs. For `TM`, we categorize the `BUMP` dataset into 16 categories depending on the target direction and the result, and generate a template for each category. Finally, `PC` uses a network of 1,600 neurons randomly connected with 160 neurons (10% probability).

3) Baseline: We design a baseline 64-core Loihi-style neuromorphic processor to evaluate the performance benefits of NeuroLobe. Although Loihi 2 is the state-of-the-art neuromorphic processor, it cannot support BCI algorithms due to its limited ISA, instruction controller, and synchronization unit (Section III-D). Therefore, we design a Loihi-style baseline (`Loihi+`) by extending the ISA and instruction controller (Section IV-A) and adopting the advanced synchronization unit (Section IV-D). Furthermore, `Loihi+` offloads neuron computation and spike processing for `NN` using a specialized datapath, as in prior Loihi variants [6], [71]. Also, it is

TABLE V
CHIP AREA COMPARISON (MM^2).

Component	NEURO	NeuroLobe
RF + ALU (Pipeline Path)	0.11	0.67
Instruction Ctrl.	-	0.56
Synchronizer	0.01	0.60
Event Queue	0.14	0.78
Router + Link	0.59	0.59
Memory Units	27.14	27.14
Total	27.99	30.34

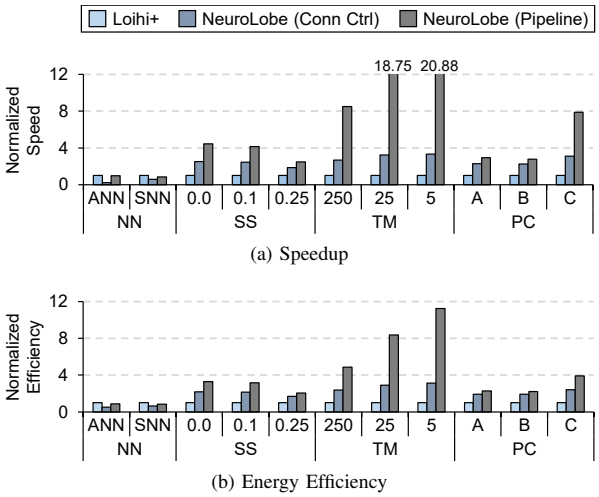
equipped with a connection controller for simple connection patterns in neural networks [26]. For fair comparisons, we set the other architectural parameters the same as those of NeuroLobe (e.g., on-chip memory capacity, interconnection topology).

We implement the CPU and GPU versions of each algorithm on quad-core ARM Cortex-A57 and 128-core Maxwell GPU embedded in the NVIDIA Jetson Nano Developer Kit as baselines. For GPU implementation of `ANN` and `SNN`, we rely on Pytorch API and Lava framework, respectively. For the other workloads, we implement custom CPU- and GPU-based frameworks that exploit the intrinsic sparsity in the datasets and parallelization opportunities. Note that our custom frameworks achieve higher performance compared to the existing open-source frameworks (i.e., Spyking Circus [144], Pytorch [99], Lava Framework [57]).

Despite the potential benefits of neuromorphic processors, ongoing debates exist regarding their efficiency compared to tensor accelerators in various applications [8], [74], [119]. Therefore, we demonstrate the efficiency of employing neuromorphic processors in the BCI context by comparing NeuroLobe against the sparse tensor accelerators. We implement two representative sparse tensor accelerators (i.e., systolic-style and vector-style) and configure them to deploy each algorithm using Sparseloop [142], [143]. The systolic-style accelerator consists of 128×128 multiply-and-add (MAC) units and a 128-way vector arithmetic unit where the processing elements share an on-chip buffer. On the other hand, the vector-style accelerator keeps 16 cores with a dedicated on-chip buffer. Each core contains eight 4-way vector MAC units, connected to a dedicated weight buffer and a 8-way vector arithmetic unit. The architectural parameters are set to enable iso-area comparisons against NeuroLobe. Both accelerators use LPDDR4 as the main memory, adopt the coordinate payload (CP) format and skipping optimizations provided by Sparseloop.

B. Low Area Overhead

We evaluate the area overhead of NeuroLobe to flexibly and efficiently support BCI algorithms. We compare the area of NeuroLobe against the neuromorphic accelerator (`NEURO`) specialized to support neural networks. The baseline resembles prior studies [5], [43] with a hardwired datapath to support neural networks and a simple synchronization mechanism (Section II-E). It does not keep an instruction memory for



(a) Speedup

(b) Energy Efficiency

Fig. 12. Evaluations on the flexibility and processing efficiency of using the fine-grained ISA, connection controller, and execution pipeline. We normalize the performance to that of `Loihi+`.

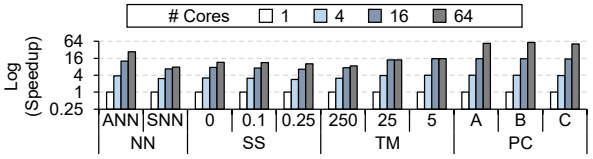
programmability and does not support multitasking. The results show that NeuroLobe incurs only an 8.39% additional area over the baseline, as on-chip memory units dominate the area in the neuromorphic processors [6], [28], [126] (Table V).

C. Effectiveness of the Key Ideas

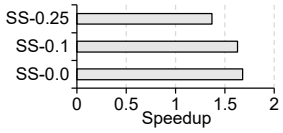
1) Flexibility and Performance: We evaluate the flexibility and performance benefits of NeuroLobe. We demonstrate the effectiveness of the flexibility optimization schemes in NeuroLobe (Table II–`Flexibility`) using `Loihi+`. In addition, we evaluate the performance enhancing techniques in NeuroLobe (Table II–`Performance`). We incrementally apply the versatile connection controller and execution pipeline to make two NeuroLobe variants: `Conn Ctrl` and `Pipeline`.

The results show that NeuroLobe enables both flexible and efficient BCI processing (Figure 12). Unlike the conventional Loihi series which only supports `NN`, `Loihi+` successfully deploys all the evaluated benchmarks by extending the ISA and synchronization units. In addition, `Conn Ctrl` improves the performance by accelerating memory address generation for all the target workloads. It achieves 2.60× and 2.25× geomean speedup and energy efficiency over `Loihi+` for non-neural network algorithms. Similarly, `Pipeline` further improves the performance to achieve 6.00× and 3.86× geomean speedup and energy efficiency over `Loihi+` non-neural network algorithms. `Pipeline` is slightly inefficient for `NN` with 1.10× slowdown and 1.17× energy inefficiency because it does not support the neuron computation stage (`NN`–❹) while `Loihi+` relies on a specialized datapath.

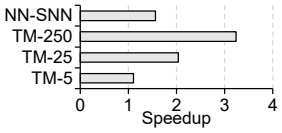
2) Scalability: We analyze the scalability benefits of NeuroLobe (Figure 13) by applying the multiphase synchronization (Section IV-D1), cascaded synchronization (Section IV-D2), and partial node insertion (Section IV-E). We evaluate the speedup of increasing the number of cores from



(a) Scalability of the phase controller without the cascaded synchronization support and partial node insertion.



(b) Speedup at 64-core scale using cascaded ack controller



(c) Speedup at 64-core scale using partial node insertion

Fig. 13. Evaluations on the scalability enhancing techniques in NeuroLobe.

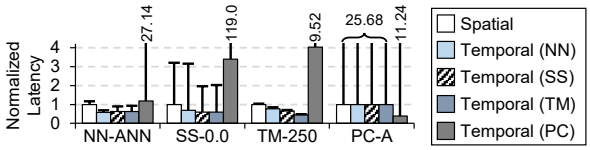
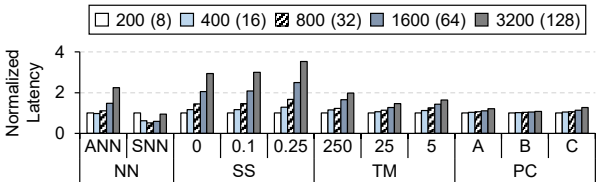


Fig. 14. The average and maximum latency of the baseline spatial multitasking and priority aware multitasking in NeuroLobe.

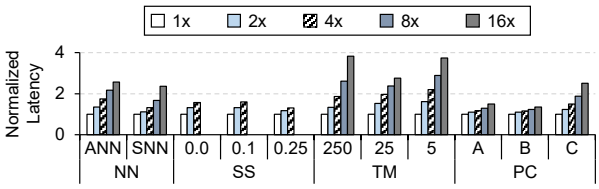
a single core to 64 cores using the multiphase synchronization unit. Then, we evaluate the speedup by eliminating redundant synchronizations using cascaded synchronization for `SS`. Lastly, we evaluate the speedup using the partial node insertion technique for `TM` and `NN-SNN` to relieve load imbalance among the cores. For `NN-SNN`, the compiler inserts partial nodes between the neurons in the hidden layer and the output layer. For `TM`, it inserts partial nodes between the neurons and templates.

The results show that the multiphase synchronization enables scalable processing with a geomean speedup of 18.49× at 64 cores (Figure 13a). In addition, the cascaded ack controller reduces the number of synchronizations to minimize the latency overhead and improve the utilization (Figure 13b). As a result, it achieves a geomean of 1.56× speedup at the 64-core scale. Lastly, using partial node insertion, NeuroLobe achieves an additional 1.84× geomean speedup by alleviating the load imbalance among the cores (Figure 13c).

3) Efficient Multitasking: Next, we analyze the benefits of the priority-aware spatio-temporal multitasking (Figure 14). We evaluate the average (bar graph) and 99th percentile latency (error bars) when multitasking four different algorithms (i.e., `NN-ANN`, `SS-0.0`, `TM-250`, and `PC-A`). We adopt various multitasking configurations, including spatial multitasking (`Spatial`) and temporal multitasking with different priorities. For example, `Temporal (NN)` refers to the case where `NN` is set to the highest priority. In `Spatial`, the NeuroLobe compiler allocates 14, 11, 9, and 30 cores to process `NN`, `SS`, `TM`, and `PC` according to the on-chip memory requirements. In `Temporal`, the compiler allocates 30 cores to process `PC` while allocating the 34 cores to process the



(a) Sensitivity to the electrode scale. The number in the parenthesis indicates the nubmer of cores. Also, the latency is normalized to that of processing 200 electrodes.



(b) Sensitivity to the peak detection rate (i.e., firing rate) at the electrodes

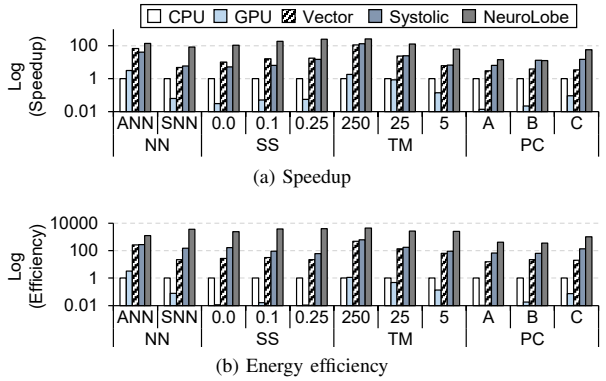
Fig. 15. Robustness of NeuroLobe with varying electrode scale and firing rate.

remaining algorithms when `PC` is not set to the highest priority (i.e., `Temporal (SS)`, `Temporal (NN)`, and `Temporal (TM)`). On the other hand, it allocates 64 cores to all the algorithms in `Temporal (PC)`. The results show that temporal multitasking effectively improves the performance of high-priority algorithms. Using `Temporal`, NeuroLobe improves the average latency and 99th percentile latency of the highest-priority algorithm by 2.05× and 1.95× on average.

D. Overall Efficiency

1) Robust BCI Processing: We evaluate the robustness of the NeuroLobe system by varying the number of neurons and firing rates in the BCI datasets (Figure 15). For each workload, we change the number of neurons (or electrodes in `SS`) from 200 to 3,200 while keeping the other parameters (e.g., connections per neuron) constant (Figure 15a). We assume a single chip-scale and increase the number of cores from 8 to 128 according to the number of neurons. This leads to slightly reduced per-core computation overhead for `NN` as the number of neurons in the hidden layer is fixed to 800. In terms of firing rates, we evaluate the workloads with up to 16× higher firing rates (Figure 15b). For `SS`, we evaluate up to 4× higher firing rates (32 Hz) as Spyking Circus does not properly support higher firing rates due to the overlapping waveforms. The results show that NeuroLobe exhibits a sub-linear increase in the latency. Even with 16× more intensive datasets, the latency increases by less than four times.

2) Comparisons Against Other Processors: We compare the speed and energy efficiency of NeuroLobe against the CPU, GPU, and sparse tensor accelerators (Figure 16). The result emphasizes the performance benefits of utilizing the memory-centric architecture and asynchronous computing nature of the neuromorphic processors. NeuroLobe achieves a geomean of 69.91× speedup and 1617.28× energy efficiency over the software baseline which selectively adopts CPU and GPU for each workload depending on the efficiency. In



(a) Speedup

(b) Energy efficiency

Fig. 16. BCI processing efficiency of NeuroLobe over the CPU, GPU, and sparse tensor accelerators. We normalize the performance to that of the CPU.

addition, NeuroLobe achieves geomean speedups of 7.00× and 5.97× and geomean energy efficiencies of 37.77× and 13.81× over the vector-style (`Vector`) and systolic-style (`Systolic`) sparse tensor accelerators, respectively.

VI. Related Work

A. Spike-Driven BCI Processors.

The existing BCI processors fail to both efficiently and flexibly deploy the spike-driven BCI algorithms. To support a variety of spike-driven BCI algorithms, existing works rely on general-purpose processors. They rely on CPUs to deploy neural networks [30], [84], [124], [134], template matching [107], [127], [131], and spike sorting [20], [75], [130]. Other works utilize GPUs [118], [134], [135] for parallel processing. However, their flexibility comes at the cost of slow speed and energy inefficiency. Some works rely on accelerators to process a specific BCI algorithm to achieve higher speed and energy efficiency. For example, Noema [2] accelerates template matching using hardwired processing units and uBrain [140] adopts a unary computing to support neural networks. However, they do not support BCI algorithms aside from template matching or neural networks. Also, they need to redesign the hardware for workloads with different configurations (e.g., different number of templates).

B. Neuromorphic Systems

Existing neuromorphic systems are tailored to support neural networks. They utilize asynchronous processing units with non-von-Neumann architectures for extremely efficient processing of ANNs and SNNs [5], [26], [27], [91], [101]. Consequently, they are adopted in various applications, including image detection [91], regression [52], and optimization [86].

Despite their high efficiency, they fail to support the entire application chains that demand a signal processing support (Section III-D). Neurobench emphasizes the importance of signal processing steps before or after executing the neural networks [145]. The signal processing potentially incurs significant costs within the overall processing pipeline. Also, the signal processing is used as a standalone application in some use cases (e.g., template matching [2], pairwise correlation [67]). NeuroLobe is the first to address this issue by rearchitecting the neuromorphic systems and demonstrating its effectiveness in the BCI domain.

VII. Conclusion

In this paper, we present NeuroLobe to flexibly and efficiently support scaled-up BCIs in the context of off-BCI spike-driven processing. The key idea is to fully exploit the event-driven computing nature of the neuromorphic processor by rearchitecting the whole system stack to deploy the spike-driven BCI algorithms. We redesign the neuromorphic processor to operate based on the extended instruction set architecture to support various BCI algorithms. Then, we design its connection controller, execution path, synchronization unit, software stack, and task controller to maximize the performance benefits. The resulting design, NeuroLobe, significantly improves the performance over the CPU, GPU, and sparse tensor accelerators.

Acknowledgment

This work was supported in part by National Research Foundation of Korea (NRF) funded by the Korean Government under Grants NRF-2021R1A2C3014131 and NRF-2019R1A5A1027055; and in part by Institute of Information & Communications Technology Planning & Evaluation (IITP) funded by the Korean Government under Grant IITP-2023-RS-2023-00256081 and IITP-2024-RS-2024-00402898. We also appreciate the support from Automation and Systems Research Institute (ASRI) and Inter-university Semiconductor Research Center (ISRC) at Seoul National University. The EDA tool was supported by the IC Design Education Center (IDEC), Korea.

References

[1] S. Abbasi, S. Maran, and D. Jaeger, "A general method to generate artificial spike train populations matching recorded neurons," *Journal of computational neuroscience*, vol. 48, 2020.

[2] A. M. Abdelhadi, E. Sha, C. Bannon, H. Steenland, and A. Moshovos, "Noema: Hardware-efficient template matching for neural population pattern detection," in *Proc. 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021.

[3] A. M. Aertsen and G. L. Gerstein, "Evaluation of neuronal connectivity: sensitivity of cross-correlation," *Brain research*, vol. 340, 1985.

[4] M. Aghagolzadeh, L. R. Hochberg, S. S. Cash, and W. Truccolo, "Predicting seizures from local field potentials recorded via intracortical microelectrode arrays," in *Proc. 38th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, 2016.

[5] F. Akopyan, J. Sawada, A. Cassidy, R. Alvarez-Icaza, J. Arthur, P. Merolla, N. Imam, Y. Nakamura, P. Datta, G.-J. Nam, B. Taba, M. Beakes, B. Brezzo, J. B. Kuang, R. Manohar, W. P. Risk, B. Jackson, and D. S. Modha, "Truenorth: Design and tool flow of a 65 mw 1 million neuron programmable neurosynaptic chip," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 34, 2015.

[6] E. Baek, H. Lee, Y. Kim, and J. Kim, "Flexlearn: fast and highly efficient brain simulations using flexible on-chip learning," in *Proc. 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2019.

[7] B. Blankertz, K.-R. Muller, G. Curio, T. M. Vaughan, G. Schalk, J. R. Wolpaw, A. Schlogl, C. Neuper, G. Pfurtscheller, T. Hinterberger, M. Schröder, and N. Birbaumer, "The bci competition 2003: progress and perspectives in detection and discrimination of eeg single trials," *IEEE transactions on biomedical engineering*, vol. 51, 2004.

[8] P. Blouw, X. Choo, E. Hunsberger, and C. Eliasmith, "Benchmarking keyword spotting efficiency on neuromorphic hardware," in *Proc. 7th annual neuro-inspired computational elements workshop (NICE)*, 2019.

[9] M. Bockbrader, "Upper limb sensorimotor restoration through brain–computer interface technology in tetraparesis," *Current Opinion in Biomedical Engineering*, vol. 11, 2019.

[10] R. B. Bod, J. Rokai, D. Meszéna, R. Fiáth, I. Ulbert, and G. Márton, "From end to end: Gaining, sorting, and employing high-density neural single unit recordings," *Frontiers in Neuroinformatics*, vol. 16, 2022.

[11] A. P. Buccino, C. L. Hurwitz, S. Garcia, J. Magland, J. H. Siegle, R. Hurwitz, and M. H. Hennig, "Spikeinterface, a unified framework for spike sorting," *Elife*, vol. 9, 2020.

[12] A. P. Buccino and G. T. Einevoll, "Mearec: a fast and customizable testbench simulator for ground-truth extracellular spiking activity," *Neuroinformatics*, vol. 19, 2021.

[13] G. Buzsáki, "Large-scale recording of neuronal ensembles," *Nature neuroscience*, vol. 7, 2004.

[14] J. Castelhano, I. Duarte, I. Bernardino, F. Pelle, S. Francione, F. Sales, and M. Castelo-Branco, "Intracranial recordings in humans reveal specific hippocampal spectral and dorsal vs. ventral connectivity signatures during visual, attention and memory tasks," *Scientific Reports*, vol. 12, 2022.

[15] M. S. Chae, Z. Yang, M. R. Yuce, L. Hoang, and W. Liu, "A 128-channel 6 mw wireless neural recording ic with spike feature extraction and uwb transmitter," *IEEE transactions on neural systems and rehabilitation engineering*, vol. 17, 2009.

[16] J. K. Chapin, K. A. Moxon, R. S. Markowitz, and M. A. Nicolelis, "Real-time control of a robot arm using simultaneously recorded neurons in the motor cortex," *Nature neuroscience*, vol. 2, 1999.

[17] J. Chen, X. Liu, H. Wu, F. Tian, W. Zou, M. Katebi, R. Eskandari, J. Yang, and M. Sawan, "A clockless robust bionic spike detector for implantable brain-computer interfaces," in *Proc. 2023 IEEE Biomedical Circuits and Systems Conference (BioCAS)*, 2023.

[18] S. Cheng and L. M. Frank, "New experiences enhance coordinated neural activity in the hippocampus," *Neuron*, vol. 57, 2008.

[19] R. H. Chowdhury, J. I. Glaser, and L. E. Miller, "Area 2 of primary somatosensory cortex encodes kinematics of the whole arm," *Elife*, vol. 9, 2020.

[20] J. E. Chung, J. F. Magland, A. H. Barnett, V. M. Tolosa, A. C. Tooker, K. Y. Lee, K. G. Shah, S. H. Felix, L. M. Frank, and L. F. Greengard, "A fully automated approach to spike sorting," *Neuron*, vol. 95, 2017.

[21] M. M. Churchland, J. P. Cunningham, M. T. Kaufman, S. I. Ryu, and K. V. Shenoy, "Cortical preparatory activity: representation of movement or first cog in a dynamical machine?" *Neuron*, vol. 68, 2010.

[22] D. Ciliberti, F. Michon, and F. Kloosterman, "Real-time classification of experience-related ensemble spiking patterns for closed-loop applications," *Elife*, vol. 7, 2018.

[23] S. C. Colachis, C. F. Dunlap, N. V. Annetta, S. M. Tamrakar, M. A. Bockbrader, and D. A. Friedenberg, "Long-term intracortical microelectrode array performance in a human: A 5 year retrospective analysis," *Journal of Neural Engineering*, vol. 18, 2021.

[24] S. C. Colachis IV, M. A. Bockbrader, M. Zhang, D. A. Friedenberg, N. V. Annetta, M. A. Schwemmer, N. D. Skomrock, W. J. Mysiw, A. R. Rezai, H. S. Bresler, and G. Sharma, "Dexterous control of seven functional hand movements using cortically-controlled transcutaneous muscle stimulation in a person with tetraplegia," *Frontiers in Neuroscience*, vol. 12, 2018.

[25] J. L. Collinger, M. A. Kryger, R. Barbara, T. Betler, K. Bowsher, E. H. Brown, S. T. Clanton, A. D. Degenhart, S. T. Foldes, R. A. Gaunt, F. E. Gyulai, E. A. Harchick, D. Harrington, J. B. Helder, T. Hemmes, M. S. Johannes, K. D. Katyal, G. S. F. Ling, A. J. C. McMorland, K. Palko, M. P. Para, J. Scheuermann, A. B. Schwartz, E. R. Skidmore, F. Solzbacher, A. V. Srikameswaran, D. P. Swanson, S. Swetz, E. C. Tyler-Kabara, M. Velliste, W. Wang, D. J. Weber, B. Wodlinger, and M. L. Boninger, "Collaborative approach in the development of high-performance brain–computer interfaces for a neuroprosthetic arm: Translation from animal models to human control," *Clinical and translational science*, vol. 7, 2014.

[26] M. Davies *et al.*, "Taking neuromorphic computing to the next level with loihi2," Intel's Neuromorphic Computing Lab, Tech. Rep., 2021.

[27] M. Davies, N. Srinivasa, T.-H. Lin, G. Chinya, Y. Cao, S. H. Choday, G. Dimou, P. Joshi, N. Imam, S. Jain, Y. Liao, C.-K. Lin, A. Lines, R. Liu, D. Mathaikutty, S. McCoy, A. Paul, J. Tse, Guruguhanathan, Venkataramanan, Y.-H. Weng, A. Wild, Y. Yang, and H. Wang, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, 2018.

[28] L. Deng, G. Wang, G. Li, S. Li, L. Liang, M. Zhu, Y. Wu, Z. Yang, Z. Zou, J. Pei, Z. Wu, X. Hu, Y. Ding, W. He, Y. Xie, and L. Shi, "Tianjic: A unified and scalable chip bridging spike-based and continuous neural computation," *IEEE Journal of Solid-State Circuits*, vol. 55, 2020.

[29] J. Dethier, V. Gilja, P. Nuyujukian, S. A. Elassaad, K. V. Shenoy, and K. Boahen, "Spiking neural network decoder for brain-machine interfaces," in *Proc. 5th International IEEE/EMBS Conference on Neural Engineering (EMBS)*, 2011.

[30] J. Dethier, P. Nuyujukian, S. I. Ryu, K. V. Shenoy, and K. Boahen, "Design and validation of a real-time spiking-neural-network decoder for brain–machine interfaces," *Journal of Neural Engineering*, vol. 10, 2013.

[31] K. DeWald, S. Pinto, A. Jois, and A. Moghaddassi, "Neural signal compression for brain-machine interface," 2023, uS Patent App. 17/690,962.

[32] J. Dragas, D. Jäckel, A. Hierlemann, and F. Franke, "Complexity optimization and high-throughput low-latency hardware implementation of a multi-electrode spike-sorting algorithm," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 23, 2014.

[33] J. Dragas, V. Viswam, A. Shadmani, Y. Chen, R. Bounik, A. Stettler, M. Radivojevic, S. Geissler, M. E. J. Obien, J. Müller, and A. Hierlemann, "In vitro multi-functional microelectrode array featuring 59 760 electrodes, 2048 electrophysiology channels, stimulation, impedance measurement, and neurotransmitter detection channels," *IEEE journal of solid-state circuits*, vol. 52, 2017.

[34] L. Duan, Z. Hongxin, M. S. Khan, and M. Fang, "Recognition of motor imagery tasks for bci using csp and chaotic pso twin svm," *The Journal of China Universities of Posts and Telecommunications*, vol. 24, 2017.

[35] B. Dutta, A. Andrei, T. Harris, C. Lopez, J. O'Callahan, J. Putzeys, B. Raducanu, S. Severi, S. Stavisky, E. Trautmann, M. Welkenhuysen, and K. Shenoy, "The neuropixels probe: A cmos based integrated microsystems platform for neuroscience and brain-computer interfaces," in *Proc. 2019 IEEE International Electron Devices Meeting (IEDM)*, 2019.

[36] J. J. Eggermont, "Properties of correlated neural activity clusters in cat auditory cortex resemble those of neural assemblies," *Journal of neurophysiology*, vol. 96, 2006.

[37] D. R. Euston, M. Tatsuno, and B. L. McNaughton, "Fast-forward playback of recent memory sequences in prefrontal cortex during sleep," *Science*, vol. 318, 2007.

[38] N. Even-Chen, D. G. Muratore, S. D. Stavisky, L. R. Hochberg, J. M. Henderson, B. Murmann, and K. V. Shenoy, "Power-saving design opportunities for wireless intracortical brain–computer interfaces," *Nature biomedical engineering*, vol. 4, 2020.

[39] D. Eytan, A. Minerbi, N. Ziv, and S. Marom, "Dopamine-induced dispersion of correlations between action potentials in networks of cortical neurons," *Journal of neurophysiology*, vol. 92, 2004.

[40] G. E. Fabiani, D. J. McFarland, J. R. Wolpaw, and G. Pfurtscheller, "Conversion of eeg activity into cursor movement by a brain–computer interface (bci)," *IEEE transactions on neural systems and rehabilitation engineering*, vol. 12, 2004.

[41] J. A. Fernandez-Leon, A. Parajuli, R. Franklin, M. Sorenson, D. J. Felleman, B. J. Hansen, M. Hu, and V. Dragoi, "A wireless transmission neural interface system for unconstrained non-human primates," *Journal of neural engineering*, vol. 12, 2015.

[42] J. Fischer, T. Milekovic, G. Schneider, and C. Mehring, "Low-latency multi-threaded processing of neuronal signals for brain-computer interfaces," *Frontiers in Neuroengineering*, vol. 7, 2014.

[43] C. Frenkel, M. Lefebvre, J.-D. Legat, and D. Bol, "A 0.086-mm^2 12.7-pj/sop 64k-synapse 256-neuron online-learning digital spiking neuromorphic processor in 28nm cmos," *IEEE Transactions on Biomedical Circuits and Systems*, vol. 13, 2018.

[44] P. Gergondet, A. Kheddar, C. Hintermüller, C. Guger, and M. Slater, "Multitask humanoid control with a brain-computer interface: user experiment with hrp-2," in *Proc. 13th International Symposium on Experimental Robotics (ISER)*, 2013.

[45] S. Gibson, J. W. Judy, and D. Markovic, "Technology-aware algorithm design for neural spike detection, feature extraction, and dimensionality reduction," *IEEE transactions on neural systems and rehabilitation engineering*, vol. 18, 2010.

[46] S. Gibson, J. W. Judy, and D. Marković, "Spike sorting: The first step in decoding the brain: The first step in decoding the brain," *IEEE Signal processing magazine*, vol. 29, 2011.

[47] Y. S. Goo, D. J. Park, J. R. Ahn, and S. S. Senok, "Spontaneous oscillatory rhythms in the degenerating mouse retina modulate retinal ganglion cell responses to electrical stimulation," *Frontiers in cellular neuroscience*, vol. 9, 2016.

[48] A. Hagberg, P. J. Swart, and D. A. Schult, "Exploring network structure, dynamics, and function using networkx," in *Proc. 7th Python in Science Conference (SciPy)*, 2008.

[49] D. Ham, H. Park, S. Hwang, and K. Kim, "Neuromorphic electronics based on copying and pasting the brain," *Nature Electronics*, vol. 4, 2021.

[50] J. Han, H. Jiang, and J. Zhu, "Neurorestoration: Advances in human brain–computer interface using microelectrode arrays," *Journal of Neurorestoratology*, vol. 8, 2020.

[51] C. Heelan, J. Komar, C. E. Vargas-Irwin, J. D. Simeral, and A. V. Nurmikko, "A mobile embedded platform for high performance neural signal computation and communication," in *Proc. 2015 IEEE Biomedical Circuits and Systems Conference (BioCAS)*, 2015.

[52] K. Henke, M. Teti, G. Kenyon, B. Migliori, and G. Kunde, "Apples-to-spikes: The first detailed comparison of lasso solutions generated by a spiking neuromorphic processor," in *Proc. 2022 International Conference on Neuromorphic Systems (ICONS)*, 2022.

[53] C. Herff, L. Diener, M. Angrick, E. Mugler, M. C. Tate, M. A. Goldrick, D. J. Krusienski, M. W. Slutzky, and T. Schultz, "Generating natural, intelligible speech from brain activity in motor, premotor, and inferior frontal cortices," *Frontiers in neuroscience*, vol. 13, 2019.

[54] G. Hong and C. M. Lieber, "Novel electrode technologies for neural recordings," *Nature Reviews Neuroscience*, vol. 20, 2019.

[55] Z. Hu, Z. Zhou, and H. Lyu, "A power-and-area-efficient channel-interleaved neural signal processor for wireless brain-computer interfaces with unsupervised spike sorting," *IEEE Transactions on Biomedical Circuits and Systems*, 2024.

[56] M. A. Huynh, T.-H. Nguyen, T.-K. Nguyen, and N.-T. Nguyen, "Convergence of implantable bioelectronics and brain–computer interfaces," *ACS Applied Electronic Materials*, vol. 5, 2023.

[57] Intel, "Lava software framework," http://lava-nc.org.

[58] S. Ito, M. E. Hansen, R. Heiland, A. Lumsdaine, A. M. Litke, and J. M. Beggs, "Extending transfer entropy improves identification of effective connectivity in a spiking cortical network model," *PloS one*, vol. 6, 2011.

[59] J. Jang, J. Lee, H. Cho, J. Lee, and H.-J. Yoo, "Wireless body-area-network transceiver and low-power receiver with high application expandability," *IEEE Journal of Solid-State Circuits*, vol. 55, 2020.

[60] I. Karageorgos, K. Sriram, J. Veselỳ, M. Wu, M. Powell, D. Borton, R. Manohar, and A. Bhattacharjee, "Hardware-software co-design for brain-computer interfaces," in *Proc. 47th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2020.

[61] M. P. Karlsson and L. M. Frank, "Awake replay of remote experiences in the hippocampus," *Nature neuroscience*, vol. 12, 2009.

[62] H. Kassiri, S. Tonekaboni, M. T. Salam, N. Soltani, K. Abdelhalim, J. L. P. Velazquez, and R. Genov, "Closed-loop neurostimulators: A survey and a seizure-predicting design example for intractable epilepsy treatment," *IEEE transactions on biomedical circuits and systems*, vol. 11, 2017.

[63] I. Käthner, S. Halder, C. Hintermüller, A. Espinosa, C. Guger, F. Miralles, E. Vargiu, S. Dauwalder, X. Rafael-Palou, M. Solà, J. M. Daly, E. Armstrong, S. Martin, and A. Kübler, "A multifunctional brain-computer interface intended for home use: an evaluation with healthy participants and potential end users with dry and gel-based electrodes," *Frontiers in Neuroscience*, vol. 11, 2017.

[64] K. Kato, M. Sawada, and Y. Nishimura, "Bypassing stroke-damaged neural pathways via a neural interface induces targeted cortical adaptation," *Nature Communications*, vol. 10, 2019.

[65] J. Kim, J. Koo, T. Kim, and J.-J. Kim, "Efficient synapse memory structure for reconfigurable digital neuromorphic hardware," *Frontiers in neuroscience*, vol. 12, 2018.

[66] S.-J. Kim, S.-H. Han, J.-H. Cha, L. Liu, L. Yao, Y. Gao, and M. Je, "A sub-μw/ch analog front-end for Δ-neural recording with spike-driven data compression," *IEEE transactions on biomedical circuits and systems*, vol. 13, 2018.

[67] R. Kobayashi, S. Kurita, A. Kurth, K. Kitano, K. Mizuseki, M. Diesmann, B. J. Richmond, and S. Shinomoto, "Reconstructing neuronal circuitry from parallel spike trains," *Nature communications*, vol. 10, 2019.

[68] D. R. Kramer, S. Kellis, M. Barbaro, M. A. Salas, G. Nune, C. Y. Liu, R. A. Andersen, and B. Lee, "Technical considerations for generating somatosensation via cortical stimulation in a closed-loop sensory/motor brain-computer interface system in humans," *Journal of Clinical Neuroscience*, vol. 63, 2019.

[69] A. Krishnan, C. Deepak, and K. Narayan, "Investigations on artificially extending the spectral range of natural vision," *APL bioengineering*, vol. 7, 2023.

[70] A. Kucyi, J. Schrouff, S. Bickel, B. L. Foster, J. M. Shine, and J. Parvizi, "Intracranial electrophysiology reveals reproducible intrinsic functional connectivity within human brain networks," *Journal of Neuroscience*, vol. 38, 2018.

[71] D. Lee, G. Lee, D. Kwon, S. Lee, Y. Kim, and J. Kim, "Flexon: A flexible digital neuron for efficient spiking neural network simulations," in *Proc. 45th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2018.

[72] H. Lee, C. Kim, Y. Chung, and J. Kim, "Neuroengine: A hardware-based event-driven simulation system for advanced brain-inspired computing," in *Proc. 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021.

[73] H. Lee, C. Kim, M. Kim, Y. Chung, and J. Kim, "Neurosync: A scalable and accurate brain simulator using safe and efficient speculation," in *Proc. 28th IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022.

[74] H. Lee, C. Kim, S. Lee, E. Baek, and J. Kim, "An accurate and fair evaluation methodology for snn-based inferencing with full-stack hardware design space explorations," *Neurocomputing*, vol. 455, 2021.

[75] J. H. Lee, D. E. Carlson, H. Shokri Razaghi, W. Yao, G. A. Goetz, E. Hagen, E. Batty, E. Chichilnisky, G. T. Einevoll, and L. Paninski, "Yass: yet another spike sorter," in *Proc. Advances in Neural Information Processing Systems 30 (NIPS)*, 2017.

[76] R. Levy, M. Taylor, and A. Sharma, "Elon musk's neuralink wins fda approval for human study of brain implants," Reuters, 2023. [Online]. Available: https://www.reuters.com/science/elon-musks-neuralink-gets-us-fda-approval-human-clinical-study-brain-implants-2023-05-25/

[77] C. M. Lewis, C. A. Bosman, and P. Fries, "Recording of brain activity across spatial scales," *Current opinion in neurobiology*, vol. 32, 2015.

[78] J.-H. Lim, Y.-W. Kim, J.-H. Lee, K.-O. An, H.-J. Hwang, H.-S. Cha, C.-H. Han, and C.-H. Im, "An emergency call system for patients in locked-in state using an ssvep-based brain switch," *Psychophysiology*, vol. 54, 2017.

[79] G. Liu, J. H. Hsiao, W. Zhou, and L. Tian, "Martmi-bci: A matlab-based real-time motor imagery brain-computer interface platform," *SoftwareX*, vol. 22, 2023.

[80] H. Liu, J. Wang, X. Chen, and J. Huang, "Neuron-aware brain-to-computer communication for wireless intracortical bci," in *Proc. 25th International Workshop on Mobile Computing Systems and Applications (HotMobile)*, 2024.

[81] X. Liu, H. Hao, L. Yang, L. Li, J. Zhang, A. Yang, and Y. Ma, "Epileptic seizure detection with the local field potential of anterior thalamic of rats aiming at real time application," in *Proc. 33rd Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, 2011.

[82] K. Louie and M. A. Wilson, "Temporally structured replay of awake hippocampal ensemble activity during rapid eye movement sleep," *Neuron*, vol. 29, 2001.

[83] S. Luan, I. Williams, M. Maslik, Y. Liu, F. De Carvalho, A. Jackson, R. Q. Quiroga, and T. G. Constandinou, "Compact standalone platform for neural recording with real-time spike sorting and data logging," *Journal of neural engineering*, vol. 15, 2018.

[84] X. Ma, F. Rizzoglio, K. L. Bodkin, E. Perreault, L. E. Miller, and A. Kennedy, "Using adversarial networks to extend brain computer interface decoding accuracy over time," *Elife*, vol. 12, 2023.

[85] J. G. Makin, J. E. O'Doherty, M. M. Cardoso, and P. N. Sabes, "Superior arm-movement decoding from cortex with a new, unsupervised-learning algorithm," *Journal of neural engineering*, vol. 15, 2018.

[86] A. R. Mangalore, G. A. Fonseca, S. R. Risbud, P. Stratmann, and A. Wild, "Neuromorphic quadratic programming for efficient and scalable model predictive control: Towards advancing speed and energy

efficiency in robotic control," *IEEE Robotics & Automation Magazine*, 2024.

[87] P. Massobrio, J. Tessadori, M. Chiappalone, and M. Ghirardi, "In vitro studies of neuronal networks and synaptic plasticity in invertebrates and in mammals using multielectrode arrays," *Neural plasticity*, 2015.

[88] C. M. McCrimmon, M. Wang, L. S. Lopes, P. T. Wang, A. Karimi-Bidhendi, C. Y. Liu, P. Heydari, Z. Nenadic, and A. H. Do, "A small, portable, battery-powered brain-computer interface system for motor rehabilitation," in *Proc. 38th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, 2016.

[89] K. P. Michmizos, D. Sakas, and K. S. Nikita, "Prediction of the timing and the rhythm of the parkinsonian subthalamic nucleus neural spikes using the local field potentials," *IEEE Transactions on Information Technology in Biomedicine*, vol. 16, 2011.

[90] J. Minguillon, M. A. Lopez-Gordo, and F. Pelayo, "Trends in eeg-bci for daily-life: Requirements for artifact removal," *Biomedical Signal Processing and Control*, vol. 31, 2017.

[91] D. S. Modha, F. Akopyan, A. Andreopoulos, R. Appuswamy, J. V. Arthur, A. S. Cassidy, P. Datta, M. V. DeBole, S. K. Esser, C. O. Otero, J. Sawada, B. Taba, A. Amir, D. Bablani, P. J. Carlson, M. D. Flickner, R. Gandhasri, G. J. Garreau, M. Ito, J. L. Klamo, J. A. Kusnitz, N. J. McClatchey, J. L. McKinstry, Y. Nakamura, T. K. Nayak, W. P. Risk, K. Schleupen, B. Shaw, J. Sivagnaname, D. F. SmithIgnacio, I. Terrizzano, and T. Ueda, "Neural inference at the frontier of energy, space, and time," *Science*, vol. 382, 2023.

[92] V. Mohan, W. P. Tay, and A. Basu, "Towards neuromorphic compression based neural sensing for next-generation wireless implantable brain machine interface," 2023, https://arxiv.org/abs/2312.09503.

[93] E. Moon, M. Barrow, J. Lim, J. Lee, S. R. Nason, J. Costello, H. S. Kim, C. Chestek, T. Jang, D. Blaauw, and J. D. Phillips, "Bridging the "last millimeter" gap of brain-machine interfaces via near-infrared wireless power transfer and data communications," *ACS photonics*, vol. 8, 2021.

[94] D. G. Muratore, P. Tandon, M. Wootters, E. Chichilnisky, S. Mitra, and B. Murmann, "A data-compressive wired-or readout for massively parallel neural recording," in *IEEE transactions on biomedical circuits and systems*, 2019.

[95] E. Musk and Neuralink, "An integrated brain-machine interface platform with thousands of channels," *Journal of medical Internet research*, vol. 21, 2019.

[96] G. O'Leary, D. M. Groppe, T. A. Valiante, N. Verma, and R. Genov, "Nurip: Neural interface processor for brain-state classification and programmable-waveform neurostimulation," *IEEE Journal of Solid-State Circuits*, vol. 53, 2018.

[97] Y. S. Park, G. R. Cosgrove, J. R. Madsen, E. N. Eskandar, L. R. Hochberg, S. S. Cash, and W. Truccolo, "Early detection of human epileptic seizures based on intracortical microelectrode array signals," *IEEE Transactions on Biomedical Engineering*, vol. 67, 2019.

[98] V. P. Pastore, P. Massobrio, A. Godjoski, and S. Martinoia, "Identification of excitatory-inhibitory links and network topology in large-scale neuronal assemblies from multi-electrode recordings," *PLoS computational biology*, vol. 14, 2018.

[99] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Proc. Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[100] F. Pei, J. Ye, D. M. Zoltowski, A. Wu, R. H. Chowdhury, H. Sohn, J. E. O'Doherty, K. V. Shenoy, M. T. Kaufman, M. Churchland, M. Jazayeri, L. E. Miller, J. Pillow, I. M. Park, E. L. Dyer, and C. Pandarinath, "Neural latents benchmark '21: Evaluating latent variable models of neural population activity," in *Proc. Neural Information Processing Systems Track on Datasets and Benchmarks 1 (NeurIPS Datasets and Benchmarks)*, 2021.

[101] J. Pei, L. Deng, S. Song, M. Zhao, Y. Zhang, S. Wu, G. Wang, Z. Zou, Z. Wu, W. He, F. Chen, N. Deng, S. Wu, Y. Wang, Y. Wu, Z. Yang, C. Ma, G. Li, W. Han, H. Li, H. Wu, R. Zhao, Y. Xie, and L. Shi, "Towards artificial general intelligence with hybrid tianjic chip architecture," *Nature*, vol. 572, 2019.

[102] P. K. D. Pramanik, N. Sinhababu, B. Mukherjee, S. Padmanaban, A. Maity, B. K. Upadhyaya, J. B. Holm-Nielsen, and P. Choudhury, "Power consumption analysis, measurement, management, and issues: A state-of-the-art review of smartphone battery and energy usage," *IEEE Access*, vol. 7, 2019.

[103] S. Qian, H. Hong, Z. Zhu, K. Chen, N. Zheng, and Y. Qi, "A high-efficiency spike sorting cloud-edge computing system with dl-dfcm," in *Proc. 2019 International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*, 2019.

[104] N. Qiao, H. Mostafa, F. Corradi, M. Osswald, F. Stefanini, D. Sumislawska, and G. Indiveri, "A reconfigurable on-line learning spiking neuromorphic processor comprising 256 neurons and 128k synapses," *Frontiers in Neuroscience*, vol. 9, 2015.

[105] Y.-L. Qin, B. L. Mcnaughton, W. E. Skaggs, and C. A. Barnes, "Memory reprocessing in corticocortical and hippocampocortical neuronal ensembles," *Philosophical Transactions of the Royal Society of London. Series B: Biological Sciences*, vol. 352, 1997.

[106] H. G. Rey, C. Pedreira, and R. Q. Quiroga, "Past, present and future of spike sorting techniques," *Brain research bulletin*, vol. 119, 2015.

[107] S. Ribeiro, D. Gervasoni, E. S. Soares, Y. Zhou, S.-C. Lin, J. Pantoja, M. Lavine, and M. A. L. Nicolelis, "Long-lasting novelty-induced neuronal reverberation during slow-wave sleep in multiple forebrain areas," *PLoS biology*, vol. 2, 2004.

[108] J. Rokai, I. Ulbert, and G. Márton, "Edge computing on tpu for brain implant signal analysis," *Neural Networks*, vol. 162, 2023.

[109] S. Ruiz, N. Birbaumer, and R. Sitaram, "Abnormal neural connectivity in schizophrenia and fmri-brain-computer interface as a potential therapeutic approach," *Frontiers in psychiatry*, vol. 4, 2013.

[110] Q. Ruoyuan, L. Tong, X. Shengwei, H. Fang, Z. Li, C. Xinxia, and D. Hua, "Neural signal acquisition and wireless transmission system design," in *Proc. 11th IEEE International Conference on Dependable, Autonomic and Secure Computing (DASC)*, 2013.

[111] U. Rutishauser, E. M. Schuman, and A. N. Mamelak, "Online detection and sorting of extracellularly recorded action potentials in human medial temporal lobe recordings, in vivo," *Journal of neuroscience methods*, vol. 154, 2006.

[112] G. Schalk, D. McFarland, T. Hinterberger, N. Birbaumer, and J. Wolpaw, "Bci2000: a general-purpose brain-computer interface (bci) system," *IEEE Transactions on Biomedical Engineering*, vol. 51, 2004.

[113] D. Seo, P. A. Merolla, and M. A. M. Osorio, "Network-on-chip for neurological data," 2022, uS Patent 11,216,400.

[114] N. Shah, S. Madugula, P. Hottowy, A. Sher, A. Litke, L. Paninski, and E. Chichilnisky, "Efficient characterization of electrically evoked responses for neural interfaces," in *Proc. Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[115] A. Shon, J.-U. Chu, J. Jung, H. Kim, and I. Youn, "An implantable wireless neural interface system for simultaneous recording and stimulation of peripheral nerve with a single cuff electrode," *Sensors*, vol. 18, 2017.

[116] J. D. Simeral, T. Hosman, J. Saab, S. N. Flesher, M. Vilela, B. Franco, J. N. Kelemen, D. M. Brandman, J. G. Ciancibello, P. G. Rezaii, E. N. Eskandar, D. M. Rosler, K. V. Shenoy, J. M. Henderson, A. V. Nurmikko, and L. R. Hochberg, "Home use of a percutaneous wireless intracortical brain-computer interface by individuals with tetraplegia," *IEEE Transactions on Biomedical Engineering*, vol. 68, 2021.

[117] R. Sitaram, A. Caria, R. Veit, T. Gaber, G. Rota, A. Kuebler, and N. Birbaumer, "Fmri brain-computer interface: a tool for neuroscientific research and treatment," *Computational intelligence and neuroscience*, 2007.

[118] N. D. Skomrock, M. A. Schwemmer, J. E. Ting, H. R. Trivedi, G. Sharma, M. A. Bockbrader, and D. A. Friedenberg, "A characterization of brain-computer interface performance trade-offs using support vector machines and deep neural networks to decode movement intent," *Frontiers in neuroscience*, vol. 12, 2018.

[119] H. Smith, J. Seekings, M. Mohammadi, and R. Zand, "Realtime facial expression recognition: Neuromorphic hardware vs. edge ai accelerators," in *Proc. 2023 International Conference on Machine Learning and Applications (ICMLA)*, 2023.

[120] K. Sriram, R. P. Pothukuchi, M. Gerasimiuk, M. Ugur, O. Ye, R. Manohar, A. Khandelwal, and A. Bhattacharjee, "Scalo: An accelerator-rich distributed system for scalable brain-computer interfacing," in *Proc. 50th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2023.

[121] S. Stanslaski, J. Herron, T. Chouinard, D. Bourget, B. Isaacson, V. Kremen, E. Opri, W. Drew, B. H. Brinkmann, A. Gunduz, T. Adamski,

G. A. Worrell, and T. Denison, "A chronically implantable neural coprocessor for investigating the treatment of neurological disorders," *IEEE transactions on biomedical circuits and systems*, vol. 12, 2018.

[122] A. Stillmaker and B. Baas, "Scaling equations for the accurate prediction of cmos device performance from 180 nm to 7 nm," *Integration*, vol. 58, 2017.

[123] J. E. Stine, I. Castellanos, M. Wood, F. Henson, F. Love, W. R. Davis, P. D. Franzon, M. Bucher, S. Basavarajaiah, J. Oh, and R. Jenkal, "Freepdk: An open-source variation-aware design kit," in *Proc. 2007 IEEE International Conference on Microelectronic Systems Education (MSE)*, 2007.

[124] D. Sussillo, P. Nuyujukian, J. M. Fan, J. C. Kao, S. D. Stavisky, S. Ryu, and K. Shenoy, "A recurrent neural network for closed-loop intracortical brain–machine interface decoders," *Journal of Neural Engineering*, vol. 9, 2012.

[125] B. E. Swinnen, M. J. Stam, A. W. Buijink, M. G. de Neeling, P. R. Schuurman, R. M. de Bie, and M. Beudel, "Employing lfp recording to optimize stimulation location and amplitude in chronic dbs for parkinson's disease: A proof-of-concept pilot study," *Deep Brain Stimulation*, vol. 2, 2023.

[126] G. Tang, K. Vadivel, Y. Xu, R. Bilgic, K. Shidqi, P. Detterer, S. Traferro, M. Konijnenburg, M. Sifalakis, G.-J. van Schaik, and A. Yousefzadeh, "Seneca: building a fully digital neuromorphic processor, design trade-offs and challenges," *Frontiers in Neuroscience*, vol. 17, 2023.

[127] M. Tatsuno, P. Lipa, and B. L. McNaughton, "Methodological considerations on the use of template matching to study long-lasting memory trace replay," *Journal of Neuroscience*, vol. 26, 2006.

[128] G. F. Tian and J. Duffin, "Spinal connections of ventral-group bulbospinal inspiratory neurons studied with cross-correlation in the decerebrate rat," *Experimental brain research*, vol. 111, 1996.

[129] E. P. Torres, E. A. Torres, M. Hernández-Álvarez, and S. G. Yoo, "Eeg-based bci emotion recognition: A survey," *Sensors*, vol. 20, 2020.

[130] P. K. Wang, S. H. Pun, C. H. Chen, E. A. McCullagh, A. Klug, A. Li, M. I. Vai, P. U. Mak, and T. C. Lei, "Low-latency single channel real-time neural spike sorting system based on template matching," *PloS one*, vol. 14, 2019.

[131] K. Watanabe, T. Haga, M. Tatsuno, D. R. Euston, and T. Fukai, "Unsupervised detection of cell-assembly sequences by similarity-based clustering," *Frontiers in Neuroinformatics*, vol. 13, 2019.

[132] J. M. Weiss, R. A. Gaunt, R. Franklin, M. L. Boninger, and J. L. Collinger, "Demonstration of a portable intracortical brain-computer interface," *Brain-Computer Interfaces*, vol. 6, 2019.

[133] J. Wessberg and M. A. Nicolelis, "Optimizing a linear algorithm for real-time robotic control using chronic cortical ensemble recordings in monkeys," *Journal of Cognitive Neuroscience*, vol. 16, 2004.

[134] F. R. Willett, D. T. Avansino, L. R. Hochberg, J. M. Henderson, and K. V. Shenoy, "High-performance brain-to-text communication via handwriting," *Nature*, vol. 593, 2021.

[135] M. S. Willsey, S. R. Nason-Tomaszewski, S. R. Ensel, H. Temmar, M. J. Mender, J. T. Costello, P. G. Patil, and C. A. Chestek, "Real-time brain-machine interface in non-human primates achieves high-velocity prosthetic finger movements using a shallow feedforward neural network decoder," *Nature Communications*, vol. 13, 2022.

[136] J. A. Wilson, "Using general-purpose graphic processing units for bci systems," in *Proc. 33rd Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, 2011.

[137] J. A. Wilson and J. Williams, "Massively parallel signal processing using the graphics processing unit for real-time brain-computer interface feature extraction," *Frontiers in Neuroengineering*, vol. 2, 2009.

[138] M. A. Wilson and B. L. McNaughton, "Reactivation of hippocampal ensemble memories during sleep," *Science*, vol. 265, 1994.

[139] A. Wohrer, M. D. Humphries, and C. K. Machens, "Population-wide distributions of neural activity during perceptual decision-making," *Progress in neurobiology*, vol. 103, 2013.

[140] D. Wu, J. Li, Z. Pan, Y. Kim, and J. S. Miguel, "ubrain: A unary brain computer interface," in *Proc. 49th Annual International Symposium on Computer Architecture (ISCA)*, 2022.

[141] Y. N. Wu, J. S. Emer, and V. Sze, "Accelergy: An architecture-level energy estimation methodology for accelerator designs," in *Proc. 2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2019.

[142] Y. N. Wu, P.-A. Tsai, A. Parashar, V. Sze, and J. S. Emer, "Sparseloop: An analytical, energy-focused design space exploration methodology for sparse tensor accelerators," in *Proc. 2021 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2021.

[143] ——, "Sparseloop: An analytical approach to sparse tensor accelerator modeling," in *Proc. 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022.

[144] P. Yger, G. L. Spampinato, E. Esposito, B. Lefebvre, S. Deny, C. Gardella, M. Stimberg, F. Jetter, G. Zeck, S. Picaud, J. Duebel, and O. Marre, "A spike sorting toolbox for up to thousands of electrodes validated with ground truth recordings in vitro and in vivo," *Elife*, vol. 7, 2018.

[145] J. Yik, K. V. den Berghe, D. den Blanken, Y. Bouhadjar, M. Fabre, P. Hueber, D. Kleyko, N. Pacik-Nelson, P.-S. V. Sun, G. Tang, S. Wang, B. Zhou, S. H. Ahmed, G. V. Joseph, B. Leto, A. Micheli, A. K. Mishra, G. Lenz, T. Sun, Z. Ahmed, M. Akl, B. Anderson, A. G. Andreou, C. Bartolozzi, A. Basu, P. Bogdan, S. Bohte, S. Buckley, G. Cauwenberghs, E. Chicca, F. Corradi, G. de Croon, A. Danielescu, A. Daram, M. Davies, Y. Demirag, J. Eshraghian, T. Fischer, J. Forest, V. Fra, S. Furber, P. M. Furlong, W. Gilpin, A. Gilra, H. A. Gonzalez, G. Indiveri, S. Joshi, V. Karia, L. Khacef, J. C. Knight, L. Kriener, R. Kubendran, D. Kudithipudi, Y.-H. Liu, S.-C. Liu, H. Ma, R. Manohar, J. M. Margarit-Taulé, C. Mayr, K. Michmizos, D. Muir, E. Neftci, T. Nowotny, F. Ottati, A. Ozcelikkale, P. Panda, J. Park, M. Payvand, C. Pehle, M. A. Petrovici, A. Pierro, C. Posch, A. Renner, Y. Sandamirskaya, C. J. Schaefer, A. van Schaik, J. Schemmel, S. Schmidgall, C. Schuman, J. sun Seo, S. Sheik, S. B. Shrestha, M. Sifalakis, A. Sironi, M. Stewart, K. Stewart, T. C. Stewart, P. Stratmann, J. Timcheck, N. Tömen, G. Urgese, M. Verhelst, C. M. Vineyard, B. Vogginger, A. Yousefzadeh, F. T. Zohora, C. Frenkel, and V. J. Reddi, "Neurobench: A framework for benchmarking neuromorphic computing algorithms and systems," 2024, https://arxiv.org/abs/2304.04640.

[146] J. Yoo and M. Shoaran, "Neural interface systems with on-device computing: Machine learning and neuromorphic architectures," *Current Opinion in Biotechnology*, vol. 72, 2021.

[147] S.-S. Yoo, T. Fairneny, N.-K. Chen, S.-E. Choo, L. P. Panych, H. Park, S.-Y. Lee, and F. A. Jolesz, "Brain-computer interface using fmri: spatial navigation by thoughts," *Neuroreport*, vol. 15, 2004.

[148] D.-Y. Yoon, S. Pinto, S. Chung, P. Merolla, T.-W. Koh, and D. Seo, "A 1024-channel simultaneous recording neural soc with stimulation and real-time spike detection," in *Proc. 2021 Symposium on VLSI Circuits (VLSIC)*, 2021.

[149] M. Zacksenhouse, M. A. Lebedev, J. M. Carmena, J. E. O'Doherty, C. Henriquez, and M. A. Nicolelis, "Cortical modulations increase in early sessions with brain-machine interface," *PloS one*, vol. 2, 2007.

[150] S. Zanos, T. P. Zanos, V. Z. Marmarelis, G. A. Ojemann, and E. E. Fetz, "Relationships between spike-free local field potentials and spike timing in human temporal cortex," *Journal of neurophysiology*, vol. 107, 2012.

[151] M. Zare, M. Nazari, A. Shojaei, M. R. Raoufy, and J. Mirnajafi-Zadeh, "Online analysis of local field potentials for seizure detection in freely moving rats," *Iranian Journal of Basic Medical Sciences*, vol. 23, 2020.

[152] M. Zhang, L. Zhang, J. H. Park, C.-W. Tsai, K. A. Ng, L. Lin, Y. Dong, J. Li, T. Tang, H. Wu, L. Wu, and J. Yoo, "A one-shot learning, online-tuning, closed-loop epilepsy management soc with 0.97 μj/classification and 97.8% vector-based sensitivity," in *Proc. 2021 Symposium on VLSI Circuits (VLSIC)*, 2021.

[153] Y. Zhou, Z. Lu, and Y. Li, "Robot control with multitasking of brain-computer interface," in *Proc. 17th International Conference on Control, Automation, Robotics and Vision (ICARCV)*, 2022.

[154] B. Zhu, M. Farivar, and M. Shoaran, "Resot: Resource-efficient oblique trees for neural signal classification," *IEEE Transactions on Biomedical Circuits and Systems*, vol. 14, 2020.